

**CORPORACIÓN UNIVERSITARIA MINUTO DE DIOS
BOGOTÁ VIRTUAL Y DISTANCIA**

PREGRADO EN INGENIERÍA DE SOFTWARE

SEMILLERO DE INTELIGENCIA ARTIFICIAL APLICADA

AGRIFORECAST: SISTEMA DE PREDICCIÓN DEL PRECIO DE LA PAPA

Modalidad: Productos de investigación (SEMILLERO)

*Documentación Técnica de Arquitectura de Software
Arquitectura MERN + Microservicio de Inteligencia Artificial (FastAPI / TensorFlow-Keras)
Despliegue en la nube sobre la plataforma Render + MongoDB Atlas
Repositorio: PDpreciopapa | Versión del documento 1.0*

Autor(s)

DORA PAOLA PACHECO MORENO

Director

DR. OMAR GUTIÉRREZ ACOSTA

Doctor en Ingeniería

BOGOTÁ, COLOMBIA

ENERO, 2026

Resumen

AgriForecast es un sistema de información diseñado para apoyar a pequeños y medianos productores de papa en Colombia mediante la predicción de precios de mercado a corto, mediano y largo plazo. Combina una aplicación web para registro de productores e inventarios con un motor de inteligencia artificial basado en redes LSTM, capaz de generar pronósticos a 1, 7 y 30 días para las variedades Negra y Criolla, acompañados de intervalos de confianza calculados con Bootstrap. El sistema responde a la alta volatilidad del precio de la papa, influenciada por factores climáticos, niveles de abastecimiento en Bogotá y Tunja, y costos de producción, que dificultan la planeación y negociación justa para los agricultores.

La solución se implementó bajo una arquitectura MERN (MongoDB, Express, React, Node.js), complementada con un microservicio de IA en FastAPI y TensorFlow/Keras, desplegada en la nube con persistencia en MongoDB Atlas.

El documento describe integralmente la arquitectura, el marco teórico, los componentes del sistema, el modelo de datos, los contratos de API, la estrategia de automatización, el despliegue en producción y el manual de uso, dejando evidencia técnica y funcional del producto de investigación desarrollado.

Palabras clave: predicción de precios, redes LSTM, inteligencia artificial, arquitectura MERN, agricultura, papa, MongoDB Atlas.

Índice

Resumen	2
Índice.....	3
1. Introducción	6
1.1 Objetivos del sistema	6
1.2 Alcance	7
1.3 Historias de Usuario (Detalladas)	7
1.4 Requisitos Funcionales (RF)	8
1.5 Requisitos No Funcionales (RNF)	9
1.6 Artefactos	10
2. Marco Teórico	15
2.1 Arquitectura MERN	15
2.2 Arquitectura Orientada a Microservicios	15
2.3 Bases de Datos NoSQL: MongoDB	16
2.4 Redes Neuronales Recurrentes y LSTM	16
2.5 Variables Exógenas y Transformación de Series	16
2.6 Cuantificación de la Incertidumbre: Bootstrap	17
2.7 Autenticación y Autorización: JSON Web Tokens (JWT)	17
2.8 Modelo de Despliegue en la Nube: Platform as a Service (PaaS)	18
3. Arquitectura General del Sistema	19
3.1 Bloques funcionales	19
3.2 Diagrama de componentes desplegados	20
3.3 Flujo de Datos Extremo a Extremo	21
4. Stack Tecnológico	23

4.1 Frontend	23
4.2 Backend (API Gateway)	23
4.3 Servicio de Inteligencia Artificial	24
4.4 Infraestructura y persistencia	24
5. Arquitectura del Motor Predictivo (Inteligencia Artificial)	26
5.1 Arquitectura base (compartida por los tres modelos)	26
5.2 Hiperparámetros por modelo / horizonte	26
5.3 Variables de entrada por producto y horizonte	27
5.4 Artefactos del modelo	27
5.5 Pipeline de inferencia	28
5.6 Evaluación de modelos	28
6. Modelo de Datos (MongoDB)	30
6.1 Colección users	30
6.2 Colección products	30
6.3 Colección predictions_history	31
7. API y Contratos de Servicio	33
7.1 Backend Node.js/Express — rutas de usuario (/users)	33
7.2 Backend Node.js/Express — rutas de producto (/products)	33
7.3 Backend Node.js/Express — rutas de predicción (/api)	33
7.4 Microservicio FastAPI	34
8. Automatización del Proceso de Predicción	35
8.1 Estrategia Adoptada	35
8.2 Flujo del Proceso Batch	35
8.3 Consideraciones de Recursos	36

9. Despliegue en Producción (Render)	37
9.1 Orden de Despliegue	37
9.2 Configuración por servicio	37
9.3 Variables de entorno clave.....	38
9.4 Integración con MongoDB Atlas	38
9.5 CORS (Cross-Origin Resource Sharing)	39
10. Consideraciones de Seguridad	40
11. Decisiones de Diseño y Trabajo Futuro	41
11.1 Decisiones de Diseño	41
11.2 Riesgos Identificados	41
11.3 Trabajo Futuro	41
12. Glosario de Términos	43
13. Funcionalidades e Interfaz del Sistema	44
13.1 Inicio de sesión y registro de usuario y recuperar contraseña	44
13.2 Home	45
13.3 Visualización de inventario y Crud de productos (Variedad de papas)	46
13.4 Historial del Modelo de Predicción (Dashboard)	48
13.5 Simulador Predictivo Avanzado (LSTM)	51
13.6 Datos (MongoDB)	55
Apéndice	57
Referencias.....	59

1. Introducción

AgriForecast es un sistema de información creado para apoyar a pequeños y medianos productores de papa en Colombia, ofreciendo predicciones de precios de mercado a 1, 7 y 30 días. Integra un aplicativo web para registro y gestión de inventarios con un motor de inteligencia artificial basado en redes LSTM, que incorpora variables externas como clima, abastecimiento y costos de producción.

El proyecto responde a la alta volatilidad del precio de la papa, que dificulta la planeación de cosechas y la negociación de precios justos. Con una arquitectura MERN (MongoDB, Express, React, Node.js) complementada por un microservicio de IA en FastAPI/TensorFlow, desplegado en la nube con persistencia en MongoDB Atlas, el sistema busca reducir la asimetría de información y ofrecer proyecciones objetivas para la toma de decisiones agrícolas.

1.1 Objetivos del sistema

- Ofrecer al agricultor predicciones del precio de la papa (variedades Negra y Criolla) a 1, 7 y 30 días, con intervalos de confianza estadísticos.
- Permitir la gestión de productos e inventario del productor (crear, editar, listar, eliminar) vinculados a su identidad (cédula).
- Automatizar la generación diaria de predicciones, evitando depender de cálculos manuales y reduciendo la latencia de consulta.
- Registrar un histórico auditable de predicciones (automáticas y simuladas) para evaluar la precisión del modelo.
- Desplegar el sistema en infraestructura *cloud* de bajo costo (Render), manteniendo separación entre frontend, backend transaccional y motor de IA.

1.2 Alcance

La documentación cubre la arquitectura completa del sistema:

- Capa de presentación: React.
- Capa de servicios: API REST en Node.js/Express.

- Capa de inteligencia artificial: microservicio FastAPI con TensorFlow/Keras.
- Capa de persistencia: MongoDB Atlas.
- Capa de infraestructura y despliegue: Render y cron-job.org.

No se detalla el proceso de entrenamiento offline de los modelos LSTM (ya entrenados y versionados como artefactos binarios), aunque sí se describe su arquitectura, hiperparámetros y pipeline de inferencia.

1.3 Historias de Usuario (Detalladas)

Estas historias incluyen **Criterios de Aceptación (CA)** para guiar las pruebas:

- **HU01 - Gestión de Seguridad y Acceso:**
 - **Descripción:** Como productor, quiero registrarme con mi cédula y una pregunta de seguridad para recuperar mi acceso si olvido la contraseña.
 - **CA1:** El sistema debe cifrar la contraseña usando **bcrypt** antes de guardarla.
 - **CA2:** La cuenta debe bloquearse automáticamente tras **3 intentos fallidos**.
- **HU02 - Control de Inventario de Cultivos:**
 - **Descripción:** Como agricultor, quiero registrar mis lotes de papa indicando variedad, cantidad y precio actual.
 - **CA1:** Solo el creador del registro (validado por **JWT**) puede editar o eliminar su producto.
- **HU03 - Consulta de Tendencias de Mercado:**
 - **Descripción:** Como usuario, quiero ver predicciones a 1, 7 y 30 días para las variedades Negra y Criolla.
 - **CA1:** Si no hay cambios en las variables, el sistema debe cargar datos precalculados de la colección `predictions_history` en menos de un segundo.
- **HU04 - Simulación Predictiva Avanzada:**
 - **Descripción:** Como analista, quiero ingresar valores personalizados de temperatura y costos para ver cómo impactan el precio futuro.
 - **CA1:** El sistema debe ejecutar 50 muestras **Bootstrap** para generar un intervalo de confianza

- al 95%

1.4 Requisitos Funcionales (RF)

Estos requisitos describen las acciones específicas que el sistema debe realizar para satisfacer las necesidades del usuario:

- **RF01 - Predicción de Precios Multivariedad:** El sistema debe generar pronósticos de precios para variedades de papa **Negra y Criolla** (específicamente en las plazas de Bogotá y Tunja).
- **RF02 - Horizontes Temporales:** El motor de IA debe calcular predicciones para tres marcos temporales distintos: **1, 7 y 30 días**.
- **RF03 - Cálculo de Incertidumbre:** El sistema debe proporcionar un **intervalo de confianza al 95%** para cada predicción puntual utilizando el método **Bootstrap**.
- **RF04 - Gestión de Inventario (CRUD):** El usuario debe poder crear, listar, actualizar y eliminar sus propios lotes de productos vinculados a su número de cédula.
- **RF05 - Gestión de Usuarios y Seguridad:** El sistema debe permitir el registro de productores, inicio de sesión mediante **JWT** y recuperación de contraseña a través de **preguntas de seguridad**.
- **RF06 - Automatización de Predicciones (Batch):** El sistema debe ejecutar diariamente un proceso automático para calcular y almacenar las predicciones de todos los modelos antes de las consultas de los usuarios.
- **RF07 - Simulador Predictivo Avanzado:** Debe permitir al usuario ingresar variables exógenas personalizadas (clima, costos) para generar una **inferencia en tiempo real**.
- **RF08 - Auditoría e Historial:** El sistema debe mantener un histórico de todas las predicciones (automáticas y manuales) para su posterior visualización y análisis de precisión.
- **RF09 - Control de Acceso:** Las cuentas de usuario deben **bloquearse automáticamente** tras 3 intentos fallidos de inicio de sesión.

1.5 Requisitos No Funcionales (RNF)

Estos requisitos definen los atributos de calidad y restricciones técnicas del sistema:

- **RNF01 - Rendimiento y Latencia:** Las consultas estándar deben ser **casi instantáneas**, resolviéndose en milisegundos mediante la lectura directa de predicciones precalculadas en la base de datos.
- **RNF02 - Disponibilidad:** La base de datos debe garantizar alta disponibilidad mediante el uso de un **replica set de 3 nodos** en MongoDB Atlas.
- **RNF03 - Escalabilidad:** La arquitectura basada en **microservicios** debe permitir escalar la lógica transaccional (Node.js) y el motor analítico (FastAPI) de forma independiente.
- **RNF04 - Seguridad de Datos:** Las contraseñas deben estar protegidas con el algoritmo de hashing **bcrypt** y las comunicaciones entre servicios deben ser seguras y estar validadas por tokens.
- **RNF05 - Interoperabilidad:** Los componentes deben comunicarse exclusivamente mediante protocolos **HTTP/HTTPS** utilizando formatos de datos **JSON**.
- **RNF06 - Portabilidad y Nube:** El sistema debe ser capaz de ejecutarse en una infraestructura de **Plataforma como Servicio (PaaS)** como Render.
- **RNF07 - Resiliencia del Proceso Batch:** Los fallos en el cálculo de una combinación específica de producto/horizonte no deben interrumpir el procesamiento de las demás tareas del lote.
- **RNF08 - Compatibilidad de Software:** El microservicio de IA debe ejecutarse específicamente en **Python 3.11** para asegurar la compatibilidad con las librerías de TensorFlow y Keras.
- **RNF09 - Usabilidad Visual:** La interfaz debe presentar la información compleja (incertidumbre, radares de escenario) de forma intuitiva mediante gráficas interactivas de **Recharts**.

1.6 Artefactos

Casos de Uso (Flujos Principales): Muestra visualmente el flujo desde que el usuario interactúa con el frontend hasta la visualización de resultados en Recharts, pasando por Node.js y el microservicio FastAPI.

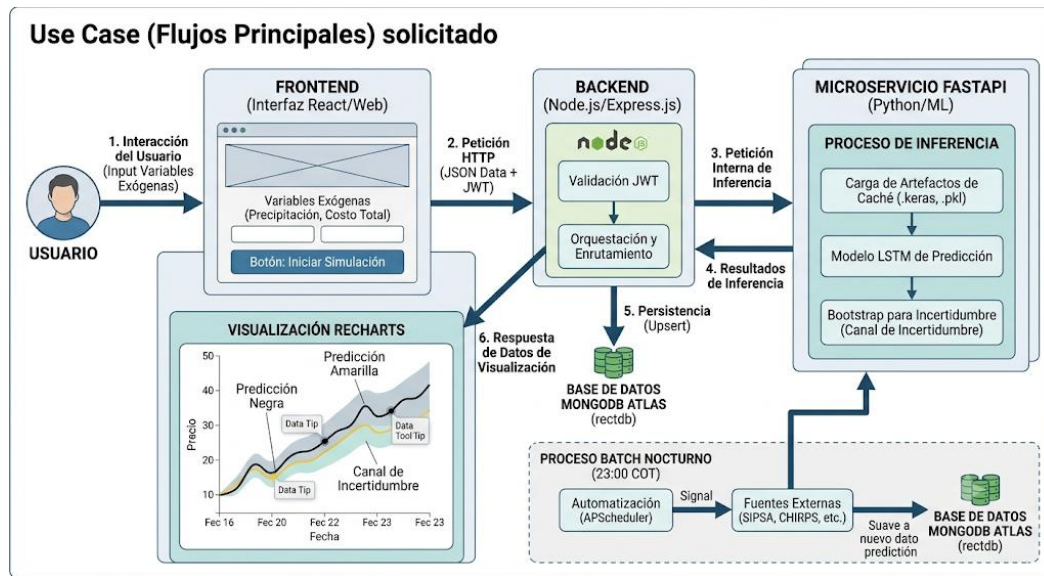


Figura 1.

Diagrama de Clases: Estructura las entidades principales (*User*, *Product*, *PredictionHistory*) con sus atributos y relaciones específicas para MongoDB Atlas.

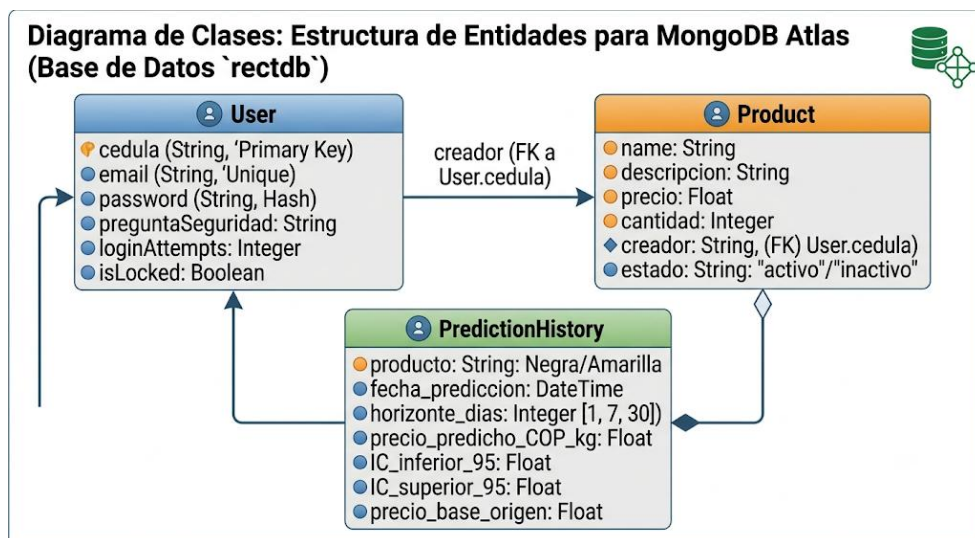


Figura 2.

Diagrama caso de uso : Este diagrama define el **alcance funcional** del sistema. Muestra que el **Productor Agrícola** (usuario final) debe **autenticarse** (include) para **gestionar productos** y **consultar predicciones** estándar o realizar **simulaciones avanzadas** (extend) con sus propios datos. Paralelamente, un **Cron-Job externo** automatiza diariamente la **actualización masiva de predicciones** para mantener el sistema al día.

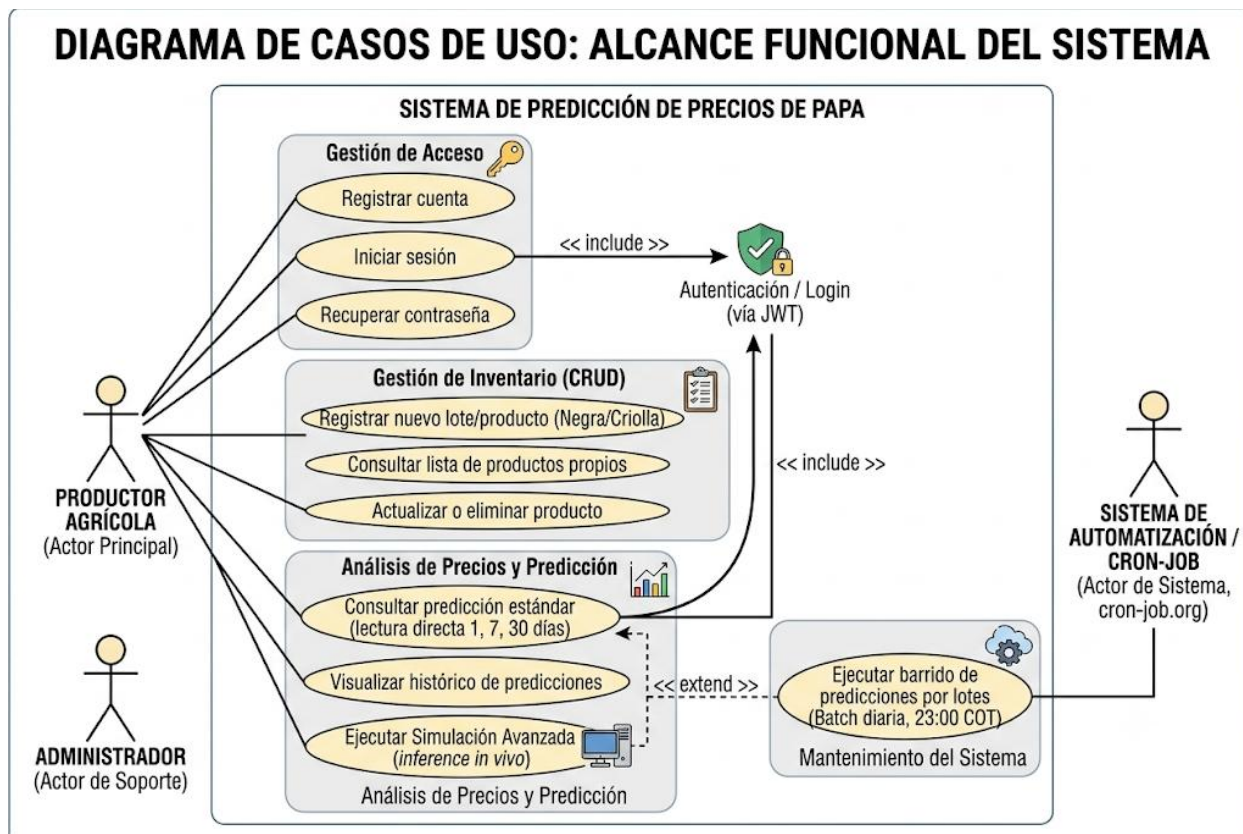


Figura 3.

Diagrama de Componentes y Control: Un gráfico circular que ilustra la lógica híbrida y la interacción entre los ocho bloques funcionales del sistema.

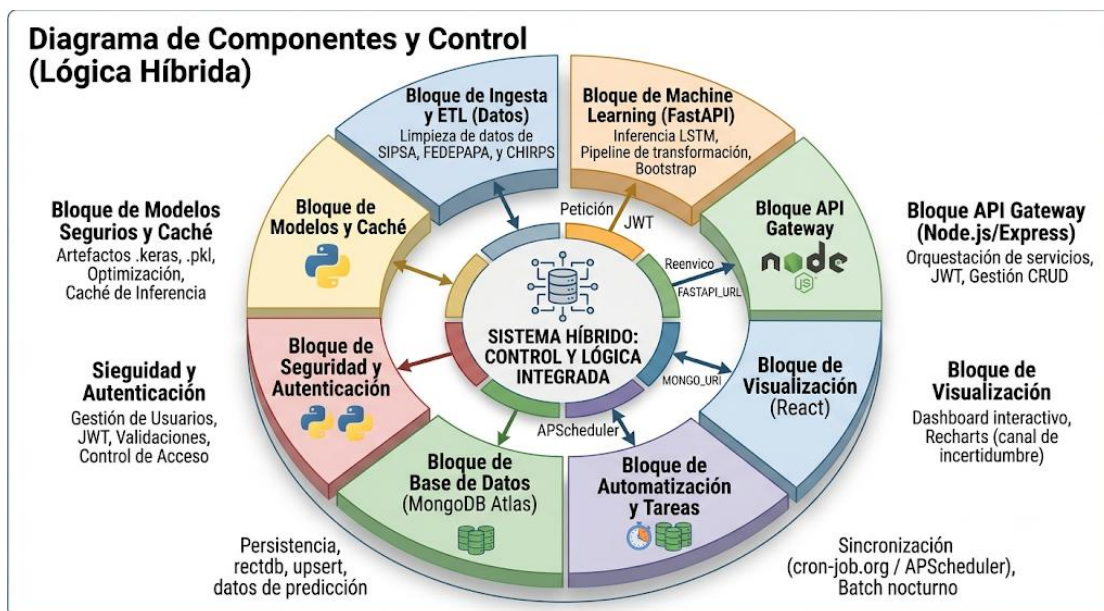


Figura 4.

Diagrama de Secuencia: Detalla cronológicamente el proceso batch diario, incluyendo el disparador externo y la orquestación con APScheduler.

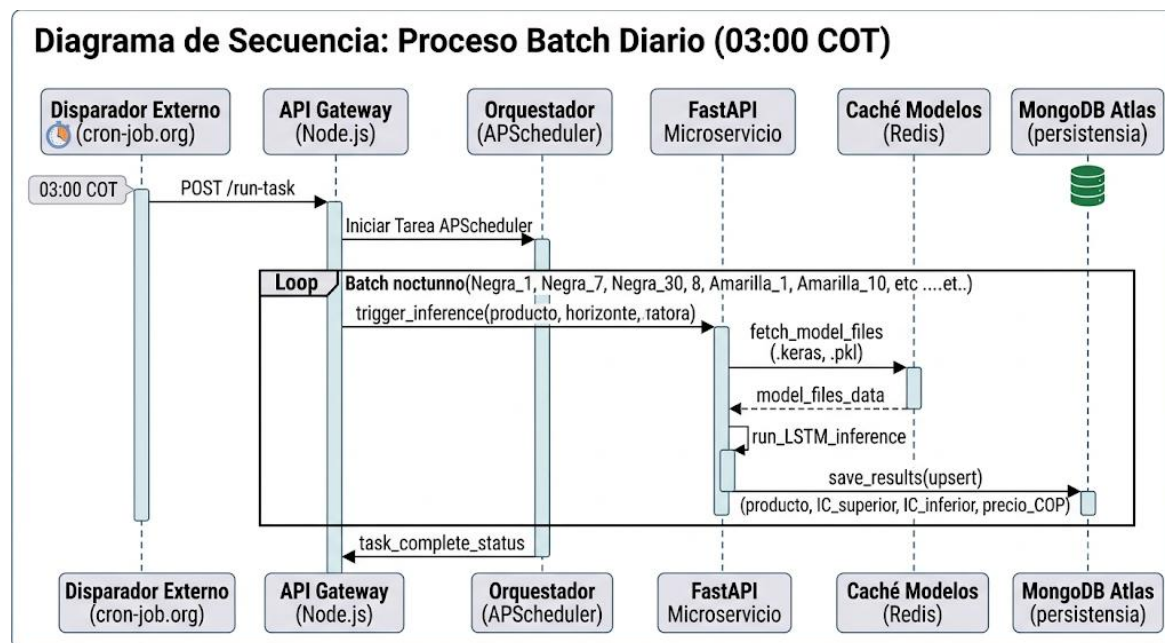


Figura 5.

Diagrama de Despliegue: Representa la infraestructura cloud en Render, mostrando las capas de cliente, lógica y analítica, y su conexión con MongoDB Atlas.

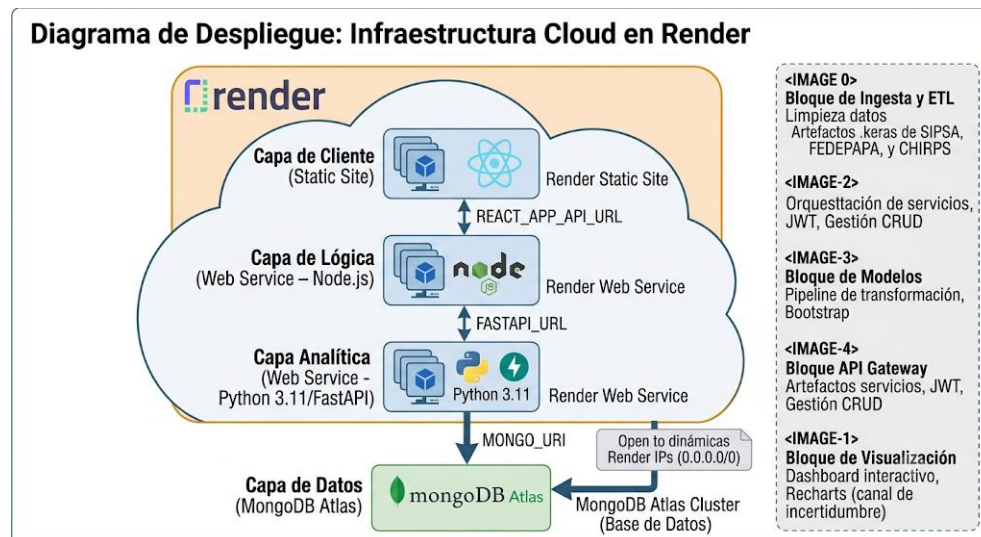


Figura 6.

Arquitectura del Motor de IA: Un esquema técnico de la red neuronal LSTM, detallando las capas, unidades y funciones de activación configuradas.}

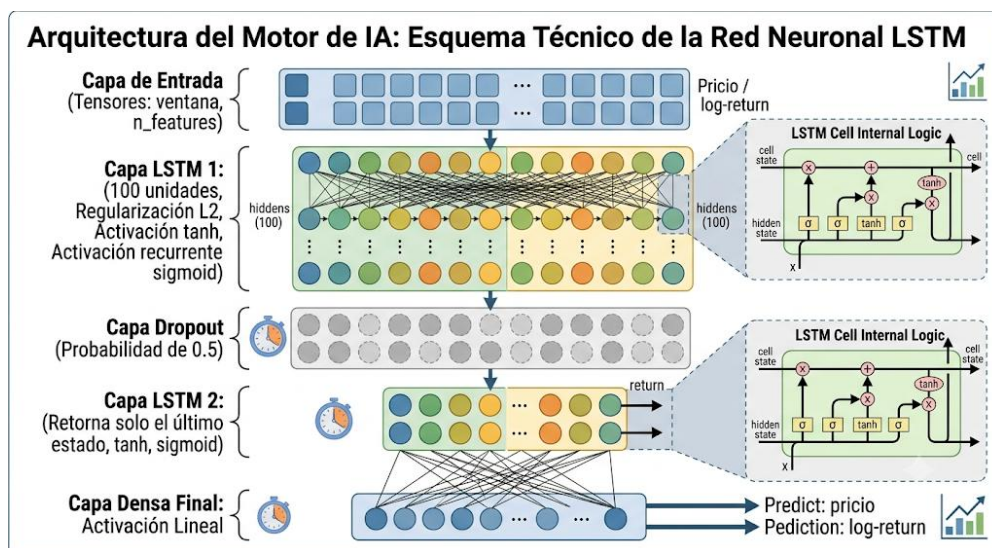


Figura 7.

2. Marco Teórico

Esta sección presenta los principios que sustentan las decisiones de diseño: el patrón arquitectónico MERN, el paradigma de microservicios, las bases de datos NoSQL orientadas a documentos, las redes neuronales recurrentes (LSTM) aplicadas a series de tiempo, el método de Bootstrap para intervalos de confianza y los modelos de despliegue en la nube (*PaaS*).

2.1 Arquitectura MERN

La pila MERN integra cuatro tecnologías JavaScript:

- MongoDB: base de datos NoSQL orientada a documentos.
- Express.js: framework minimalista para construir APIs REST sobre Node.js.
- React: biblioteca declarativa para interfaces de usuario basadas en componentes.
- Node.js: entorno de ejecución de JavaScript en servidor.

Su principal ventaja es la homogeneidad del lenguaje (JavaScript/JSX) en todas las capas, lo que facilita compartir estructuras de datos en JSON sin traducciones adicionales.

En AgriForecast, la pila MERN se complementa con un microservicio en Python/FastAPI para la inferencia del modelo de IA, ya que el ecosistema de *Machine Learning* (TensorFlow, Keras, NumPy, Pandas, scikit-learn) es nativo de Python. Esto configura una arquitectura híbrida MERN + microservicio de IA, comunicados mediante peticiones HTTP internas.

2.2 Arquitectura Orientada a Microservicios

El sistema aplica el principio de separación de responsabilidades, dividiendo la lógica en servicios independientes:

- Servicio de presentación: aplicación React compilada en archivos estáticos (HTML/CSS/JS).
- Servicio transaccional (API Gateway): backend Node.js/Express para autenticación, CRUD de productos y orquestación de peticiones al motor de IA.
- Servicio de inferencia (ML Service): microservicio FastAPI que expone el modelo LSTM mediante HTTP.

Esta separación permite escalar cada servicio de forma independiente y aplicar el principio de aislamiento de fallos: un error en un servicio no colapsa el sistema completo.

2.3 Bases de Datos NoSQL: MongoDB

MongoDB almacena información en documentos tipo JSON (internamente BSON), agrupados en colecciones. A diferencia de las bases relacionales, no requiere un esquema rígido, lo que facilita la evolución del modelo de datos.

El proyecto usa Mongoose como *Object Document Mapper (ODM)* en Node.js, que provee validación de esquemas y seguridad de tipos. La base productiva se despliega en MongoDB Atlas, con un *replica set* de tres nodos para garantizar alta disponibilidad.

2.4 Redes Neuronales Recurrentes y LSTM

La predicción de precios agrícolas es un problema de series de tiempo. Las RNN mantienen un estado oculto que captura dependencias entre observaciones pasadas y presentes, pero sufren del problema del gradiente desvaneciente.

Las LSTM (Hochreiter & Schmidhuber, 1997) solucionan este problema con una celda de memoria regulada por tres compuertas: olvido, entrada y salida. Estas permiten retener información relevante en secuencias largas.

AgriForecast utiliza una LSTM apilada de dos capas, con:

- Primera capa LSTM que retorna la secuencia completa.
- Segunda capa LSTM que retorna solo el último estado.
- Capas *Dropout* para regularización.
- Capa densa final con activación lineal para el precio (o log-retorno).

2.5 Variables Exógenas y Transformación de Series

Además de la serie histórica del precio, el modelo incorpora variables externas:

- Temperatura media y su rezago a 20 días.
- Precipitación acumulada a 30 días.
- Abastecimiento en toneladas.

- Costo total de producción.

Cuando la serie no es estacionaria, se aplica la transformación de log-retornos:

Esto estabiliza la varianza y permite reconstruir el precio original aplicando la exponenciación inversa.

2.6 Cuantificación de la Incertidumbre: Bootstrap

Toda predicción puntual conlleva incertidumbre. Para cuantificarla sin asumir distribuciones paramétricas, el sistema usa Bootstrap:

- Se generan $N=50$ muestras sintéticas perturbando la ventana de entrada con ruido gaussiano proporcional a la desviación estándar histórica.
- Se ejecuta la inferencia sobre cada muestra.
- Se construye la distribución empírica de las predicciones.

El intervalo de confianza al 95 % se obtiene con los percentiles 2.5 y 97.5, lo que resulta más robusto que asumir simetría gaussiana, especialmente en modelos con transformaciones logarítmicas.

2.7 Autenticación y Autorización: JSON Web Tokens (JWT)

El control de acceso del backend se basa en JSON Web Tokens (JWT), un estándar abierto (RFC 7519) para transmitir información de forma segura como objetos JSON firmados digitalmente.

- Tras un inicio de sesión exitoso, el servidor emite un token firmado que incluye el identificador del usuario, su correo y su cédula, válido por 24 horas.
- Cada petición a rutas protegidas (ej. gestión de productos) debe incluir el token en la cabecera Authorization, validado por un *middleware* antes de permitir el acceso.
- Las contraseñas se almacenan con bcrypt, un algoritmo de hashing adaptativo diseñado para resistir ataques de fuerza bruta.

2.8 Modelo de Despliegue en la Nube: Platform as a Service (PaaS)

El sistema se despliega bajo el modelo Platform as a Service (PaaS), utilizando la plataforma Render. En este esquema, el proveedor gestiona servidores, sistema operativo, *runtime* y balanceo de carga, mientras el equipo de desarrollo solo define el código fuente, el comando de construcción (*build command*) y el de arranque (*start command*).

Render distingue tres tipos de servicio:

- Static Site: para el frontend compilado.
- Web Service: para procesos de larga duración con un puerto HTTP expuesto (backend y microservicio de IA).
- Cron Job: para tareas programadas puntuales.

Una característica relevante del plan gratuito de Render es la suspensión por inactividad (cold start): si un Web Service no recibe tráfico en 15 minutos, se detiene y la siguiente petición debe esperar entre 30 y 60 segundos para reactivarse. Esta limitación condicionó la estrategia de automatización del sistema (sección 7), sustituyendo el Cron Job nativo de pago de Render por un disparador HTTP externo gratuito (cron-job.org).

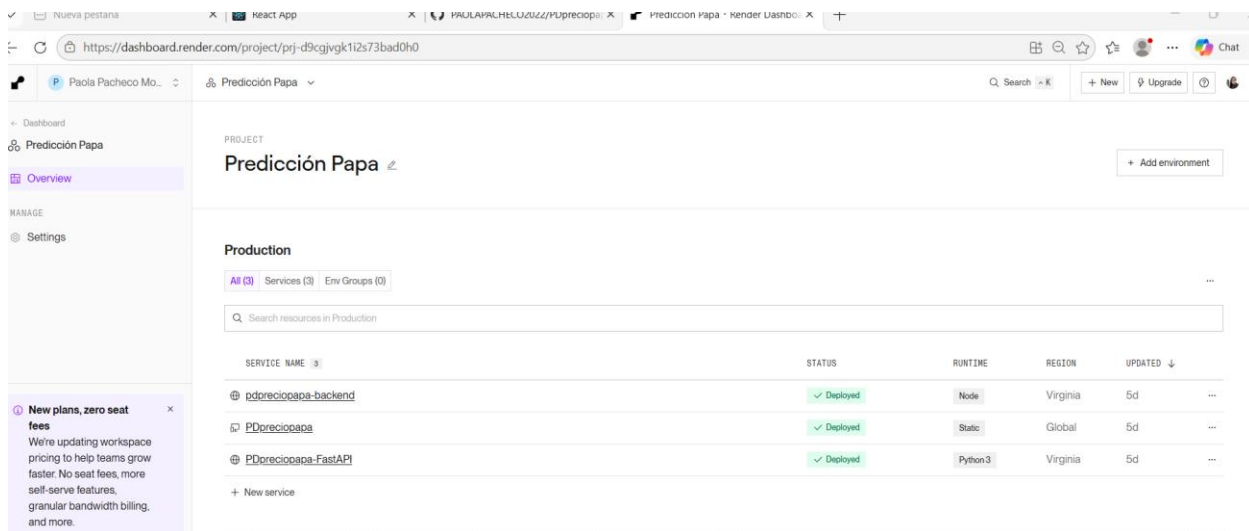


Figura 8.

3. Arquitectura General del Sistema

La arquitectura de AgriForecast se organiza en ocho bloques funcionales que conforman un flujo de datos de extremo a extremo: desde la ingesta de fuentes de datos históricas hasta la visualización final por parte del usuario. La siguiente figura resume dicho flujo.



Figura 9. Arquitectura del sistema de predicción del precio de la papa (fuentes de datos, ETL, features, modelos, evaluación, API, frontend y usuarios finales).

3.1 Bloques funcionales

- **Fuentes de datos:** SIPSA–FEDEPAPA (precios históricos y abastecimiento), datos climáticos tipo CHIRPS (precipitación, temperatura), costos de producción (insumos, mano de obra, transporte) y otras fuentes (DANE / variables económicas).
- **Procesamiento (ETL):** limpieza de datos, eliminación de duplicados y atípicos, integración de datasets por fecha y región, transformación temporal (conversión a frecuencia diaria/semanal) y validación de calidad.

- Ingeniería de features: normalización (MinMaxScaler / StandardScaler según el modelo), construcción de variables de entrada (precio, abastecimiento, costos, precipitación, temperatura) y definición de subconjuntos de variables por horizonte de predicción.
- Modelos: ocho modelos LSTM entrenados (una combinación de producto × horizonte), con arquitectura base compartida y regularización L2, almacenados como artefactos .keras junto con sus escaladores .pkl.
- Evaluación: métricas de error MAE, RMSE y R^2 , utilizadas en la fase de entrenamiento/selección de modelos para comparar arquitecturas e hiperparámetros.
- API y servicios: microservicio FastAPI que expone el endpoint de inferencia (/predict) y el backend Node.js/Express que actúa como API Gateway hacia el frontend y hacia la base de datos.
- Frontend (dashboard): aplicación React con visualizaciones de precios históricos, predicciones y análisis/reportes.
- Usuarios: agricultores, comerciantes y analistas/entidades del sector, cada uno consumiendo la información según su rol.

3.2 Diagrama de componentes desplegados

En producción, el sistema se materializa en cuatro servicios desplegados de forma independiente en Render, más un servicio externo de programación de tareas, comunicados como se describe a continuación:

Tabla 1. Componentes desplegados en producción

Componente	Tecnología	Tipo de servicio en Render	Responsabilidad
Frontend	React 18 (Create React App)	Static Site	Interfaz de usuario, consumo de la API vía Axios
Backend / API Gateway	Node.js + Express + Mongoose	Web Service	Autenticación JWT, CRUD de productos, orquestración de predicciones, lectura/escritura en MongoDB Atlas

Componente	Tecnología	Tipo de servicio en Render	Responsabilidad
Servicio de IA (ML Service)	Python + FastAPI + TensorFlow/Keras	Web Service	Inferencia del modelo LSTM, cálculo de Bootstrap, tarea programada interna (APScheduler)
Base de datos	MongoDB Atlas (replica set, 3 nodos)	Servicio gestionado externo	Persistencia de usuarios, productos e histórico de predicciones
Disparador de automatización	cron-job.org	Servicio externo gratuito	Ejecuta un POST diario contra el backend para disparar el batch de predicciones

Nota: Elaboración propia.

La comunicación entre los distintos módulos del sistema se realiza exclusivamente vía HTTP/HTTPS:

- El frontend consume la API del backend mediante peticiones REST autenticadas con JWT.
- El backend se comunica con el microservicio de IA a través de peticiones internas (Axios) cuando se requiere una simulación con variables exógenas personalizadas.
- Tanto el backend como el microservicio de IA se conectan de forma independiente al mismo clúster de MongoDB Atlas, compartiendo la base de datos *rectdb*.

3.3 Flujo de Datos Extremo a Extremo

1. El microservicio de IA ejecuta automáticamente la inferencia para cada combinación de producto (papa negra, papa criolla-Bogotá, papa criolla-Tunja) y horizonte (1, 7 y 30 días), almacenando los resultados en la colección *predictions_history* de MongoDB Atlas.
2. Cuando el agricultor abre el panel de predicción sin modificar variables exógenas, el backend consulta directamente esa colección (último registro por fecha) y devuelve la respuesta en milisegundos, sin invocar al modelo en tiempo real.
3. Si el agricultor activa el modo de “simulación avanzada” e introduce variables climáticas o de costos personalizadas, el backend reenvía la petición al microservicio de IA (`GET`

`/predict`), ejecuta la inferencia en tiempo real y guarda también ese resultado en la misma colección, enriqueciendo el histórico con escenarios hipotéticos.

4. El frontend renderiza la respuesta (precio predicho, intervalo de confianza al 95 %, fecha objetivo) en gráficas interactivas con Recharts: canal de incertidumbre, radar de escenario, velocímetro de precisión y dona de margen de ganancia.
5. El módulo de histórico (`PredictionHistoryDashboard`) permite auditar todas las predicciones almacenadas, filtrando por producto, horizonte y fecha de ejecución.

4. Stack Tecnológico

4.1 Frontend

Tabla 2. Stack tecnológico del frontend

Tecnología	Versión / uso
React	^18.2.0 — biblioteca de componentes
react-scripts (Create React App)	5.0.1 — toolchain de build (webpack, babel)
react-router-dom	^5.3.4 — enrutamiento SPA (Switch/Route/PrivateRoute)
react-bootstrap + bootstrap	^2.10 / ^5.3 — sistema de componentes UI
recharts	^3.9 — gráficas (AreaChart, RadarChart, RadialBarChart, PieChart)
chart.js + react-chartjs-2	^4.5 / ^5.3 — gráficas complementarias
axios	^1.x — cliente HTTP con interceptor de token JWT
lucide-react	iconografía
sweetalert2	alertas/modales

Nota: Elaboración propia.

4.2 Backend (API Gateway)

Tabla 3. Stack tecnológico del backend (API Gateway)

Tecnología	Versión / uso
Node.js + Express	^4.22 — servidor HTTP y enrutamiento REST
Mongoose	^7.8 — ODM sobre MongoDB
mongodb (driver nativo)	^7.5
jsonwebtoken	^9.0 — emisión y verificación de JWT
bcrypt	^6.0 — hashing de contraseñas
cors	^2.8 — control de orígenes permitidos

Tecnología	Versión / uso
dotenv	^17.4 — variables de entorno
mongoose-unique-validator	^4.0 — validación de unicidad (cédula, email)
node-cron	^4.6 — programación opcional de tareas internas

Nota: Elaboración propia.

4.3 Servicio de Inteligencia Artificial

Tabla 4. Stack tecnológico del servicio de inteligencia artificial

Tecnología	Uso
Python 3.11 (fijado vía PYTHON_VERSION)	runtime — 3.11 es compatible con TensorFlow; versiones ≥3.13 no lo son
FastAPI	framework web asíncrono para exponer el endpoint /predict
Uvicorn	servidor ASGI de producción
TensorFlow / Keras	carga (.keras) e inferencia de los modelos LSTM
NumPy / Pandas	manipulación numérica y de series de tiempo
joblib	carga de los escaladores (.pkl) entrenados
APScheduler (BackgroundScheduler)	tarea programada interna (cron diario 23:00) dentro del ciclo de vida de FastAPI
PyMongo (MongoClient)	conexión directa e independiente a MongoDB Atlas para persistir predicciones

Nota: Elaboración propia.

4.4 Infraestructura y persistencia

Tabla 5. Infraestructura y persistencia

Componente	Uso
MongoDB Atlas	clúster gestionado (replica set de 3 nodos), base de datos rectdb
Render — Static Site	hospedaje del frontend compilado (build de React)
Render — Web Service x2	hospedaje del backend Node.js y del microservicio FastAPI
cron-job.org	disparo HTTP externo diario hacia el endpoint de predicción por lotes
GitHub	control de versiones y disparador de despliegue continuo en Render

Nota: Elaboración propia.

5. Arquitectura del Motor Predictivo (Inteligencia Artificial)

El núcleo analítico del sistema es un conjunto de ocho redes neuronales LSTM entrenadas de forma independiente, una por cada combinación válida de producto (variedad de papa) y horizonte de predicción. Todas comparten la misma arquitectura base, variando únicamente sus hiperparámetros y el subconjunto de variables de entrada.

5.1 Arquitectura base (compartida por los tres modelos)



5.2 Hiperparámetros por modelo / horizonte

Los modelos LSTM implementados presentan configuraciones diferenciadas según variedad y horizonte:

- Papa Negra (h = 1 y 7): dos capas LSTM de 100 y 60 unidades, con regularización L2 = 0.02, ventana de 60 días y escalador estándar.
- Papa Negra (h = 30, override): arquitectura ajustada con 100 y 80 unidades, ventana reducida a 30 días, escalador estándar con log-retorno y misma regularización.
- Papa Amarilla Bogotá (h = 1, 7, 30): dos capas de 32 unidades, regularización L2 = 0.03, ventana de 60 días, escalador estándar y diferenciación aplicada.
- Papa Amarilla Tunja (h = 1 y 7): dos capas de 33 y 32 unidades, sin regularización L2, ventana de 30 días, escalador estándar y diferenciación aplicada.

5.3 Variables de entrada por producto y horizonte

- papa_negra — h=1: precio_promedio + prec30_mm (2 variables).
- papa_negra — h=7: precio_promedio + tmedia_c_lag20 + prec30_mm (3 variables).
- papa_negra — h=30: precio_promedio + Cant_Ton_Total + costo_total + tmedia_c (4 variables).
- papa_amarilla_BOGOTA (todos los horizontes): precio_promedio + Cant_Ton_Total + costo_total + tmedia_c_lag20 (4 variables).
- papa_amarilla_TUNJA (todos los horizontes): precio_promedio + Cant_Ton_Total + costo_total + tmedia_c (4 variables).

Cuando el horizonte agrega varios días en un solo bloque temporal (modo «agregado»), cada variable se resume mediante una operación específica: last (último valor, típicamente el precio), sum (acumulados, como abastecimiento y costo) o mean (promedios, como temperatura y precipitación).

5.4 Artefactos del modelo

Cada modelo entrenado se guarda en tres archivos dentro del directorio `ml_service/model_artifacts/`:

- `best_<producto>_h<horizonte>.keras`: contiene la arquitectura y los pesos de la red neuronal.
- `scaler_full_<producto>.pkl`: escalador aplicado a todas las variables de entrada.
- `scaler_target_<producto>.pkl`: escalador aplicado únicamente a la variable objetivo (precio), usado para devolver la predicción a su escala original.

Para optimizar el rendimiento, el microservicio mantiene una caché en memoria (MODEL_CACHE) que carga cada modelo y sus escaladores una sola vez por combinación producto/horizonte, evitando lecturas repetidas desde disco durante el ciclo de vida del proceso.

5.5 Pipeline de inferencia

- Se construye (o recupera) la ventana temporal de entrada más reciente, con un histórico mínimo igual al tamaño de ventana (`window_size`) definido por el modelo.

- Si el modelo requiere diferenciación logarítmica, la variable objetivo se transforma a log-retornos antes de aplicar el escalado.
- Se aplica el escalador (`scaler_full`) y se seleccionan únicamente las columnas relevantes para el horizonte solicitado.
- Si el modo de secuencia es “agregado” y el horizonte es mayor a 1, los datos se agrupan en bloques de tamaño h , aplicando la operación definida por variable (último valor, suma o promedio).
- Se ejecuta `model.predict()` sobre el tensor resultante y se invierte el escalado con `scaler_target`; si hubo diferenciación logarítmica, se reconstruye el precio multiplicando por el precio base observado y aplicando la función exponencial.
- Se ejecuta el procedimiento Bootstrap (50 muestras con ruido gaussiano proporcional a la desviación histórica de cada variable cruda), reproduciendo los pasos 3–5 en cada muestra perturbada, para estimar el intervalo de confianza al 95 % mediante percentiles.

5.6 Evaluación de modelos

Durante la fase de entrenamiento/selección (offline, no expuesta como servicio en producción) se emplean las siguientes métricas de error para comparar arquitecturas e hiperparámetros:

- MAE (Mean Absolute Error): error absoluto medio entre el precio predicho y el observado, en las mismas unidades del precio (COP/kg).
- RMSE (Root Mean Squared Error): raíz del error cuadrático medio, penaliza en mayor medida los errores grandes.
- R^2 (Coeficiente de determinación): proporción de la varianza del precio explicada por el modelo.

6. Modelo de Datos (MongoDB)

La base de datos rectdb aloja tres colecciones principales, mapeadas mediante esquemas de Mongoose en el backend y, en el caso de `predictions_history`, escritas también directamente desde el microservicio de Python mediante PyMongo. Esta doble vía de escritura (Node.js y Python) sobre la misma colección es posible gracias a que MongoDB no impone un esquema rígido a nivel de motor de base de datos; la consistencia estructural se garantiza por convención entre ambos servicios.

6.1 Colección users

Tabla 7. Estructura de la colección users

Campo	Tipo	Descripción
nombres, apellidos	String	Datos de identificación del productor
cedula	Number (único)	Identificador nacional, clave de negocio del usuario
celular, telefono, direccion	String	Datos de contacto
departamento, municipio	String	Ubicación geográfica
email	String (único)	Credencial de acceso
password	String	Hash bcrypt (nunca se serializa en las respuestas JSON)
preguntaSeguridad / respuestaSeguridad	String	Mecanismo de recuperación de contraseña sin correo electrónico
loginAttempts	Number	Contador de intentos fallidos consecutivos
isLocked	Boolean	Bloqueo de cuenta tras 3 intentos fallidos

Nota: Elaboración propia.

6.2 Colección products

Tabla 8. Estructura de la colección products

Campo	Tipo	Descripción
name, descripcion	String	Identificación del producto/lote
precio, cantidad	Number	Datos comerciales del lote
fecha	Date	Fecha de registro
estado	String (enum)	activo / inactivo
creador	Number	Cédula del propietario (validación de dueño en updateProduct/deleteProduct)

Nota: Elaboración propia.

6.3 Colección predictions_history

Tabla 9. Estructura de la colección predictions_history

Campo	Tipo	Descripción
producto	String	papa_negra / papa_amarilla_BOGOTA / papa_amarilla_TUNJA
fecha_ejecucion	Date	Instante (UTC) en que se calculó la predicción
fecha_prediccion	Date	Fecha objetivo a la que corresponde el pronóstico
horizonte_dias	Number	1, 7 o 30
precio_predicho_COP_kg	Number	Precio central estimado
IC_inferior_95 / C_superior_95	Number	Límites del intervalo de confianza (percentiles 2.5/97.5 del Bootstrap)
n_bootstrap_validas	Number	Cantidad de muestras Bootstrap finitas utilizadas
precio_base_origen	Number	Precio de referencia usado para reconstruir el valor en modelos con diferenciación

Nota: Elaboración propia.

La escritura sobre esta colección utiliza la operación upsert (update_one con upsert=True), indexada lógicamente por la tripleta (producto, fecha_prediccion, horizonte_dias), evitando duplicados cuando el proceso automático se re-ejecuta para la misma fecha objetivo.

7. API y Contratos de Servicio

7.1 Backend Node.js/Express — rutas de usuario (/users)

Tabla 10. Rutas de usuario (/users)

o	Métod	Ruta	Descripción
	POST	/users/register	Registro de un nuevo productor
	POST	/users/login	Inicio de sesión; emite JWT; bloquea la cuenta al tercer intento fallido
	POST	/users/get-question	Paso 1 de recuperación de contraseña (por cédula)
	POST	/users/reset-password	Paso 2 de recuperación (valida respuesta de seguridad y reasigna contraseña)
	GET	/users/:id	Consulta de usuario por cédula
	PUT	/users/update/:id	Actualización de perfil

Nota: Elaboración propia.

7.2 Backend Node.js/Express — rutas de producto (/products)

CRUD estándar protegido por el middleware de autenticación JWT (auth.js): getAllProducts, getProductById, createProduct (asocia automáticamente la cédula del token como creador), updateProduct y deleteProduct (ambas validan que el usuario autenticado sea el propietario del recurso antes de modificarlo o eliminarlo).

7.3 Backend Node.js/Express — rutas de predicción (/api)

Tabla 11. Rutas de producto (/products)

Método	Ruta	Descripción
GET	/api/agro-predictions	Controlador inteligente: si la petición no incluye variables exógenas, consulta directamente MongoDB Atlas (predicción precalculada); si las incluye, reenvía la solicitud al microservicio FastAPI en tiempo real y persiste el resultado.

Método	Ruta	Descripción
GET	/api/predictions-history	Listado histórico filtrable por producto, horizonte_días y fechaPrediccion (rango de día completo en UTC), ordenado por fecha_ejecucion descendente, con límite configurable.

Nota: Elaboración propia.

7.4 Microservicio FastAPI

Tabla 12. Rutas de predicción (/api)

o	Métod	Ruta	Descripción
	GET	/predict	Ejecuta la inferencia LSTM + Bootstrap para un producto y horizonte dados, con variables exógenas opcionales; responde precio_predicho_COP_kg, IC_inferior_95, IC_superior_95 y n_bootstrap_validas.
	POST	/run-task	Dispara manualmente el barrido completo (todos los productos × todos los horizontes) y su persistencia en MongoDB Atlas — usado para pruebas y como gancho del disparador externo de automatización.
	GET	/docs	Documentación interactiva autogenerada (Swagger UI) provista por FastAPI.

Nota: Elaboración propia.

Todas las respuestas del microservicio de IA siguen un formato JSON homogéneo, reutilizado sin transformación por el backend de Node.js al reenviar la respuesta al frontend, lo que reduce el acoplamiento entre ambos servicios.

8. Automatización del Proceso de Predicción

El sistema debe ofrecer predicciones casi instantáneas al usuario, sin que este espere el proceso completo de inferencia + Bootstrap (50 ejecuciones por combinación producto/horizonte). Para lograrlo, se desacopla el cálculo de la consulta mediante un proceso batch programado.

8.1 Estrategia Adoptada

Se evaluaron dos alternativas:

- Cron Job nativo de Render: descartado, porque el plan gratuito no lo incluye (solo disponible desde el plan Starter).
- Planificador interno del microservicio (APScheduler): se inicializa un *BackgroundScheduler* en FastAPI que programa la función `tarea_prediccion_automatica()` para ejecutarse diariamente a las 23:00. Esta fue la opción principal, complementada con un disparador externo para evitar problemas si el servicio está “dormido” por inactividad.

Como salvaguarda ante el cold start del plan gratuito de Render, se añadió un tercer mecanismo: un servicio externo gratuito (cron-job.org) que envía cada día una petición POST al backend a las 03:00 (hora de Colombia). Esto asegura la ejecución del batch y, además, “despierta” el servicio si estaba suspendido.

8.2 Flujo del Proceso Batch

1. Se construye o recupera el conjunto de datos recientes (histórico mínimo requerido por cada modelo).
2. Se recorren los productos configurados (papa negra, papa criolla-Bogotá, papa criolla-Tunja) y, para cada uno, sus horizontes válidos (1, 7, 30 días según el caso).
3. Se cargan los artefactos del modelo (con caché), se ejecuta la inferencia + Bootstrap y se construye el documento de predicción.
4. El documento se guarda en *predictions_history* mediante upsert, evitando duplicados para la misma fecha objetivo.

5. Los errores en una combinación producto/horizonte se registran en el log, sin interrumpir el procesamiento de las demás (resiliencia parcial del batch).

8.3 Consideraciones de Recursos

La carga de TensorFlow y de los ocho modelos LSTM en memoria exige bastante RAM. En el plan gratuito de Render (512 MB), esto puede provocar errores por falta de memoria (*Out Of Memory*, código 137) si se cargan demasiados modelos a la vez.

La caché de artefactos (MODEL_CACHE) mitiga parcialmente este riesgo reutilizando modelos ya cargados entre ejecuciones. Sin embargo, la recomendación para producción es migrar el microservicio de IA a un plan con mayor memoria dedicada.

9. Despliegue en Producción (Render)

El repositorio del proyecto (PDpreciopapa) es un monorepo con tres subdirectorios desplegados de forma independiente:

- `Cliente/` → frontend React.
- `Servidor/` → backend Node.js.
- `Servidor/ml_service/` → microservicio FastAPI.

Render permite declarar un Root Directory distinto para cada servicio dentro del mismo repositorio, evitando fragmentar el proyecto en múltiples repositorios.

9.1 Orden de Despliegue

El orden recomendado y seguido fue:

1. Backend Node.js.
2. Microservicio FastAPI.
3. Frontend React.

Esto se debe a que:

- El frontend necesita conocer la URL pública del backend en tiempo de compilación (inyectada como variable de entorno).
- El backend necesita conocer la URL del microservicio de IA para reenviar las peticiones de simulación avanzada.

9.2 Configuración por servicio

Tabla 13. Configuración de despliegue por servicio

Servicio	Root Directory	Build Command	Start Command
Frontend (Static Site)	Cliente	npm install && npm run build	— (publish directory: build)
Backend (Web Service)	Servidor	npm install	node index.js

Servicio	Root Directory	Build Command	Start Command
ML Service (Web Service)	Servidor/ml_service	pip install -r requirements.txt	uvicorn prediction_service:app --host 0.0.0.0 --port \$PORT

Nota: Elaboración propia.

9.3 Variables de entorno clave

Tabla 14. Variables de entorno clave

Servicio	Variable	Propósito
Backend	PORT	Puerto inyectado dinámicamente por Render (no se fija manualmente)
Backend	FASTAPI_URL	URL base del microservicio de IA (sin ruta /predict incluida en la variable, para evitar duplicación de segmentos en la URL final)
Backend	MONGO_URI (en database/db.js)	Cadena de conexión al clúster de MongoDB Atlas
ML Service	PYTHON_VERSION	Fija la versión de Python (3.11.x) compatible con TensorFlow, sobrescribiendo el valor por defecto de Render
ML Service	MODELS_DIR (resuelto en código)	Ruta absoluta calculada a partir de __file__ para localizar model_artifacts/ independientemente del directorio de trabajo del proceso
Frontend	REACT_APP_API_URL / VITE_API_URL	URL pública del backend, inyectada en tiempo de build (Create React App usa el prefijo REACT_APP_)

Nota: Elaboración propia.

9.4 Integración con MongoDB Atlas

En el plan gratuito de Render las direcciones IP salientes no son estáticas, por lo que la lista blanca de acceso del clúster de MongoDB Atlas se configuró para permitir conexiones desde cualquier origen (0.0.0.0/0). La seguridad de la base de datos depende exclusivamente

de las credenciales de usuario y contraseña de la cadena de conexión, cuya confidencialidad es crítica. Estas credenciales se gestionan mediante variables de entorno, nunca en el código fuente público del repositorio.

9.5 CORS (Cross-Origin Resource Sharing)

Como el frontend y el backend son servicios independientes con dominios distintos (ej. `https://pdpreciopapa.onrender.com` y `https://pdpreciopapa-backend.onrender.com`), el backend debe declarar explícitamente el origen del frontend como permitido mediante el *middleware* cors de Express. También se incluyen los orígenes de desarrollo local (`localhost:3000`). Sin esta configuración, el navegador bloquearía las peticiones por la política de mismo origen.

10. Consideraciones de Seguridad

- Contraseñas: nunca se almacenan en texto plano; se aplica hashing con `bcrypt` y *salting* automático antes de guardar el usuario, mediante un *middleware* `pre('save')` de Mongoose.
- Control de fuerza bruta: bloqueo de cuenta (`isLocked`) tras 3 intentos fallidos de inicio de sesión, requiriendo intervención de un administrador o recuperación por pregunta de seguridad.
- Autorización por propietario: las operaciones de edición/eliminación de productos verifican que la cédula del token JWT coincida con el creador del recurso, evitando modificaciones cruzadas.
- Serialización segura: el método `toJSON` del esquema de usuario elimina los campos sensibles (contraseña y respuesta de seguridad) de cualquier respuesta HTTP.
- Gestión de secretos: las cadenas de conexión y llaves JWT se externalizan como variables de entorno, no como literales en el código.
- CORS restringido: solo se permiten orígenes conocidos, evitando políticas abiertas (*) para reducir riesgos de ataques desde dominios no autorizados.

11. Decisiones de Diseño y Trabajo Futuro

11.1 Decisiones de Diseño

- Separación del cálculo (batch nocturno) de la consulta (lectura directa en MongoDB), garantizando tiempos de respuesta bajos en el frontend.
- Persistencia dual desde Node.js y Python sobre la misma colección, aprovechando la flexibilidad de MongoDB.
- Modo híbrido de consulta en `/api/agro-predictions` (lectura de caché vs. inferencia en vivo), reutilizando el mismo contrato de API para consultas rápidas y simulaciones avanzadas.
- Uso de `upsert` en la escritura del histórico, haciendo el proceso batch idempotente ante reejecuciones o fallos parciales.

11.2 Riesgos Identificados

- Uso de datos simulados en el pipeline de inferencia en producción; se recomienda sustituirlos por consultas reales a la colección de precios históricos.
- Restricciones de memoria del plan gratuito de Render frente al costo computacional de TensorFlow, que pueden interrumpir el servicio de IA.
- Dependencia de un servicio externo (`cron-job.org`) para garantizar la ejecución diaria del batch, lo que introduce un punto de fallo fuera del control directo.

11.3 Trabajo Futuro

- Sustituir la generación sintética de históricos por fuentes de datos reales y versionadas (SIPSA, CHIRPS, DANE).
- Incorporar un panel de evaluación continua (backtesting) que compare predicciones históricas con precios reales observados.
- Migrar la automatización a un Cron Job nativo o a una cola de tareas dedicada, a medida que el proyecto escale.

- Versionar formalmente los modelos LSTM (registro de experimentos) para trazabilidad entre el modelo desplegado y sus métricas de evaluación (MAE, RMSE, R^2).

12. Glosario de Términos

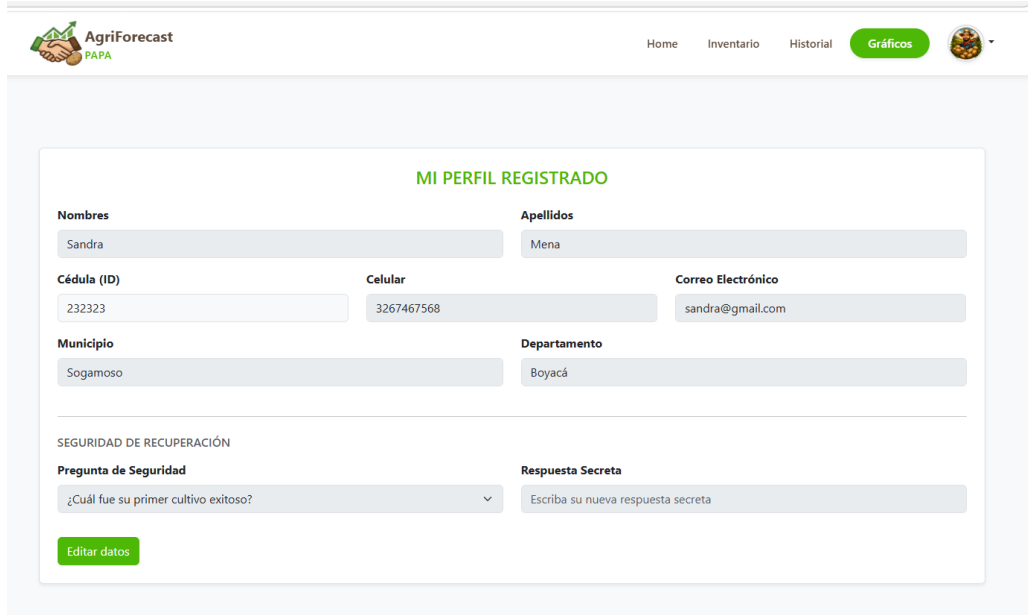
Tabla 15. Glosario de términos

Término	Definición
API REST	Interfaz de programación de aplicaciones basada en el protocolo HTTP y los principios de la arquitectura Representational State Transfer.
LSTM	Long Short-Term Memory: arquitectura de red neuronal recurrente con memoria de largo plazo, mediante compuertas de olvido/entrada/salida.
Bootstrap	Método estadístico de remuestreo utilizado para estimar la distribución de un estimador sin asumir una forma paramétrica.
JWT	JSON Web Token: estándar para representar afirmaciones (claims) de forma segura entre dos partes mediante un objeto JSON firmado.
ODM	Object Document Mapper: capa de abstracción que mapea documentos de una base de datos NoSQL a objetos del lenguaje de programación (Mongoose para MongoDB).
PaaS	Platform as a Service: modelo de computación en la nube donde el proveedor gestiona la infraestructura y el runtime.
Upsert	Operación de base de datos que actualiza un documento si existe, o lo inserta si no existe.
Cold start	Retardo de arranque que sufre un servicio en la nube tras un período de inactividad, al reactivarse para atender una nueva petición.
CORS	Cross-Origin Resource Sharing: mecanismo de seguridad del navegador que restringe las peticiones HTTP entre distintos orígenes (dominios).
MAE / RMSE / R^2	Métricas de error para evaluar modelos de regresión: error absoluto medio, raíz del error cuadrático medio y coeficiente de determinación, respectivamente.

Nota: Elaboración propia.

13. Manual de Usuario: Funcionalidades e Interfaz del Sistema

13.1 Inicio de sesión y registro de usuario y recuperar contraseña



AgriForecast PAPA Home Inventario Historial Gráficos

MI PERFIL REGISTRADO

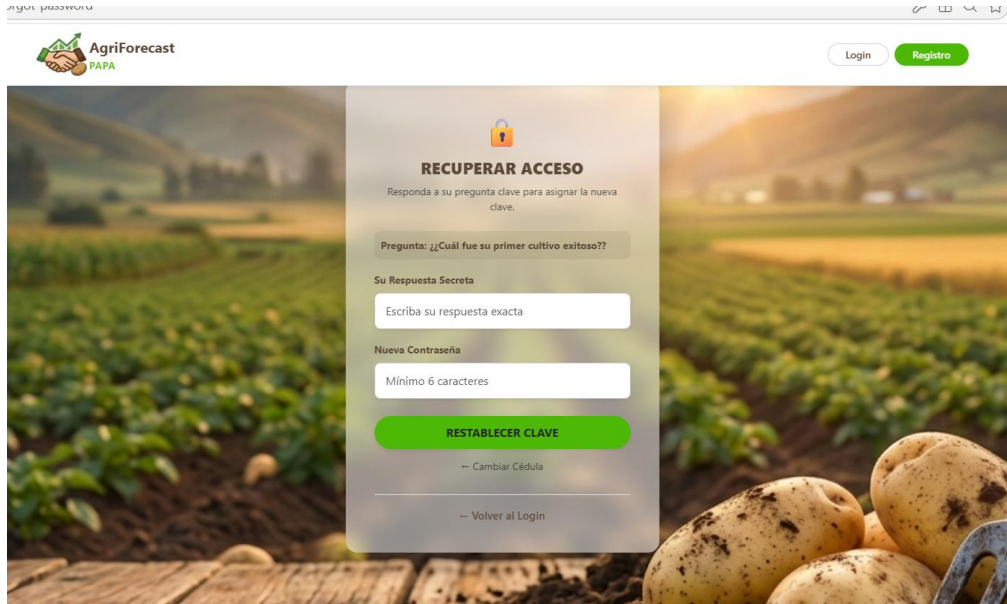
Nombres	Apellidos	
Sandra	Mena	
Cédula (ID)	Celular	Correo Electrónico
232323	3267467568	sandra@gmail.com
Municipio	Departamento	
Sogamoso	Boyacá	

SEGURIDAD DE RECUPERACIÓN

Pregunta de Seguridad	Respuesta Secreta
¿Cuál fue su primer cultivo exitoso?	Escriba su nueva respuesta secreta

[Editar datos](#)

Figura 10.



AgriForecast PAPA Login Registro

RECUPERAR ACCESO

Responda a su pregunta clave para asignar la nueva clave.

Pregunta: ¿Cuál fue su primer cultivo exitoso??

Su Respuesta Secreta

Escriba su respuesta exacta

Nueva Contraseña

Mínimo 6 caracteres

[RESTABLECER CLAVE](#)

[Cambiar Cédula](#)

[Volver al Login](#)

Figura 11.

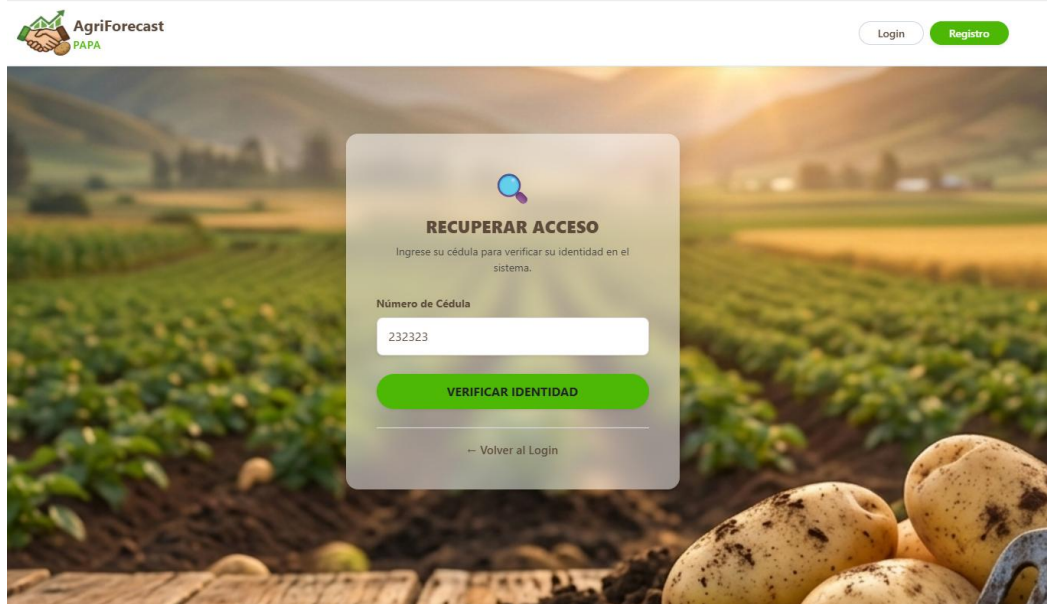


Figura 12.

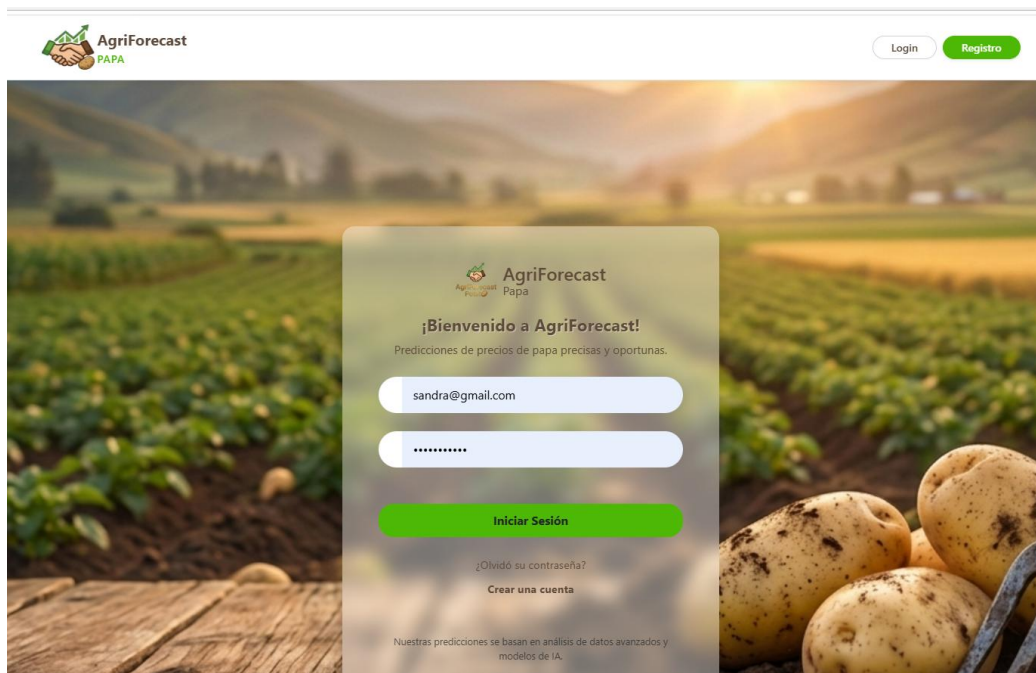


Figura 13.

13.2 Home

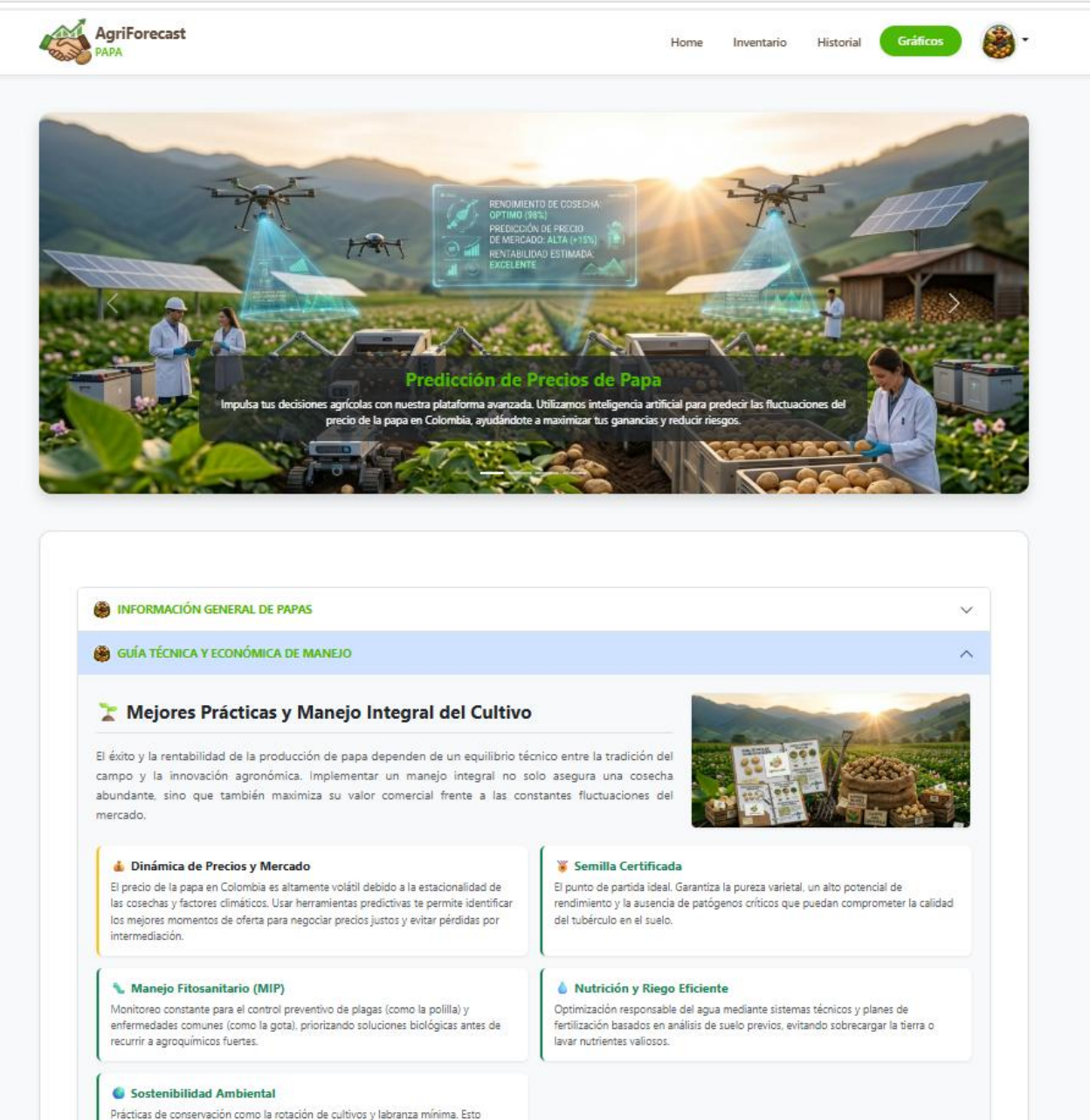


Figura 14.

13.3 Visualización de inventario y Crud de productos (Variedad de papas)

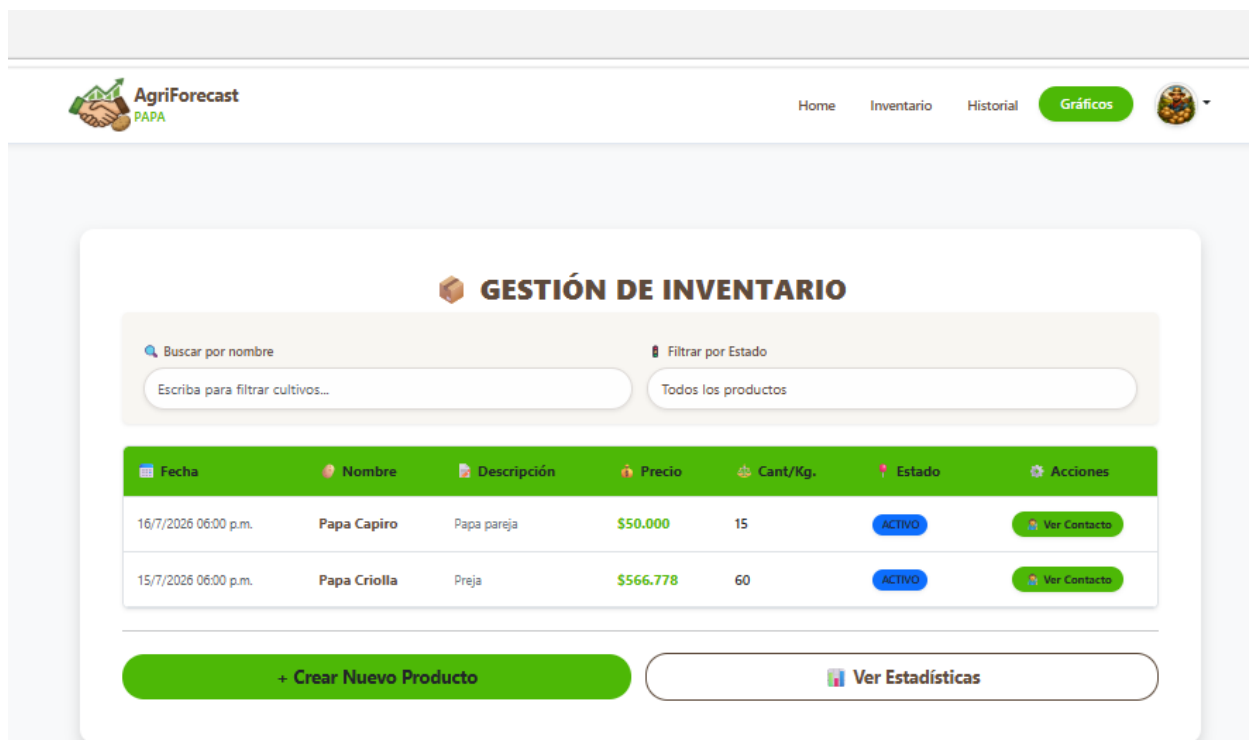


Figura 15.

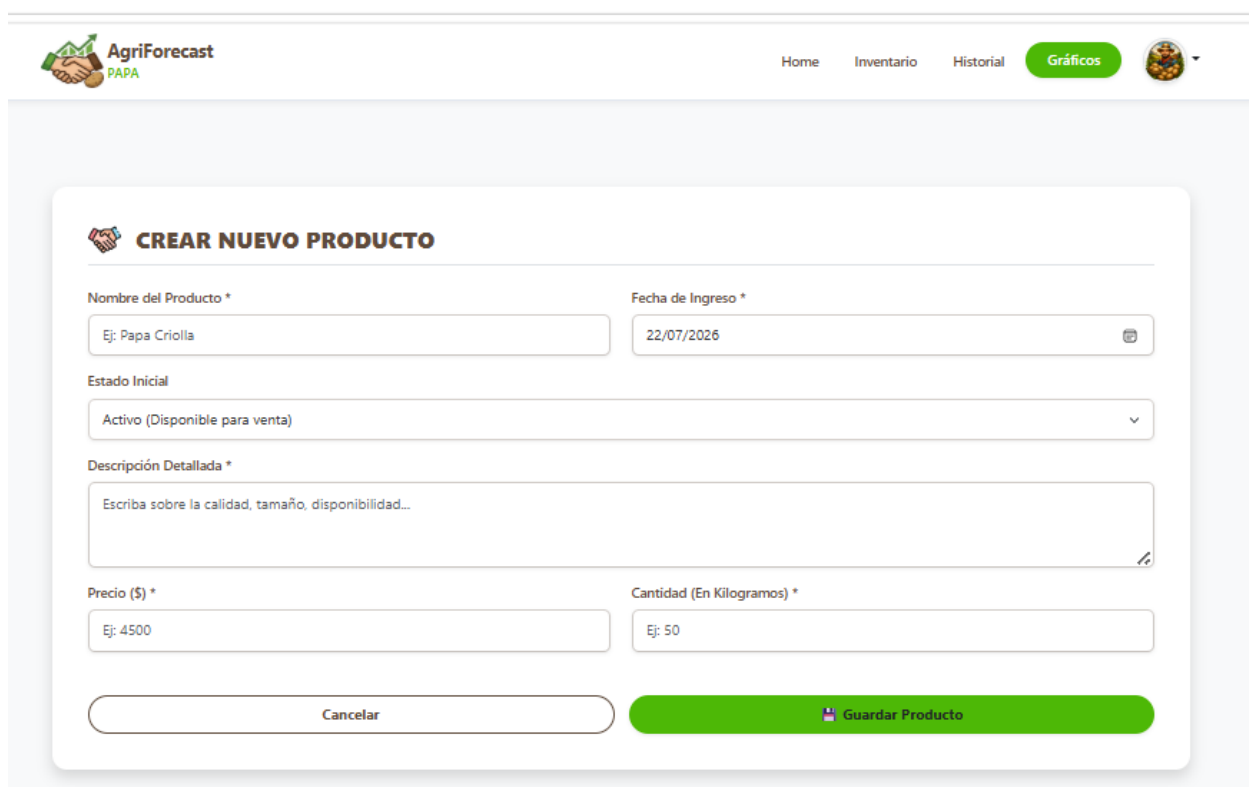
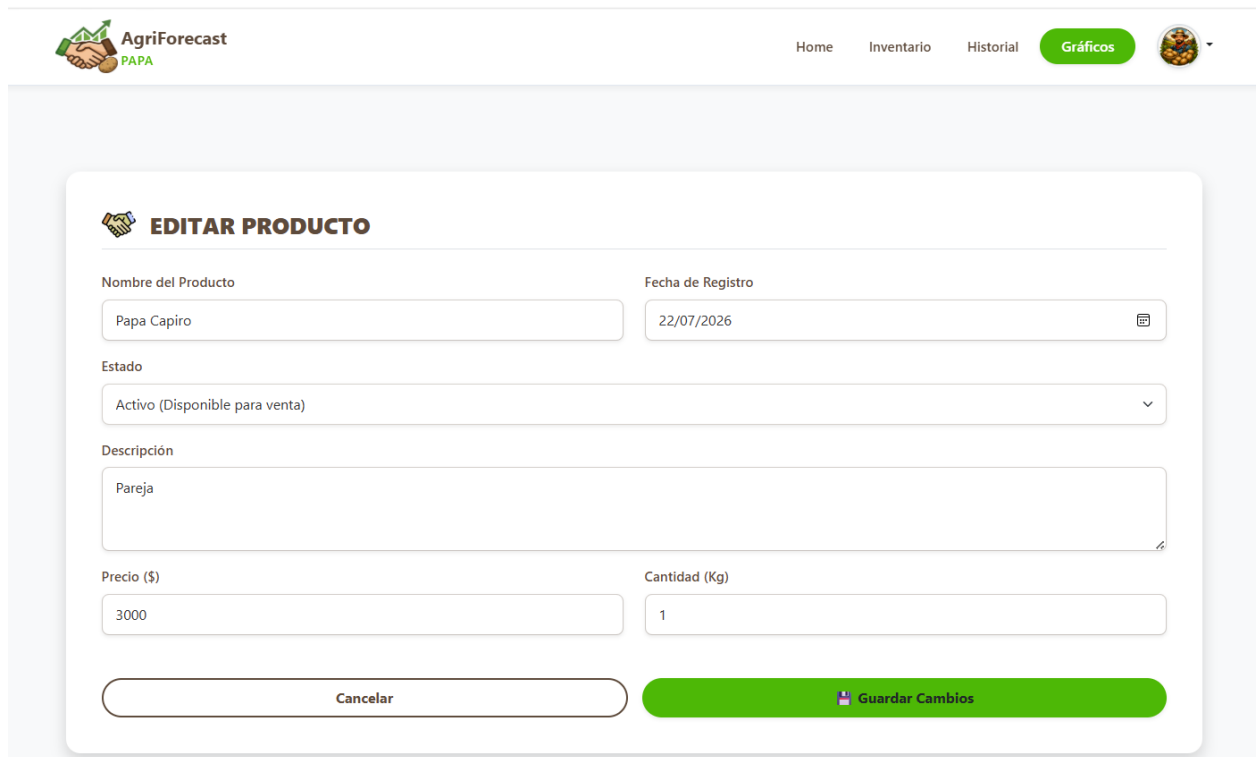


Figura 16.



The screenshot shows the 'EDITAR PRODUCTO' (Edit Product) form in the AgriForecast PAPA system. The form is titled 'EDITAR PRODUCTO' with a small icon of a handshake. It contains several input fields and a dropdown menu. The 'Nombre del Producto' field is filled with 'Papa Capiro'. The 'Fecha de Registro' field is filled with '22/07/2026'. The 'Estado' dropdown menu is set to 'Activo (Disponible para venta)'. The 'Descripción' field is filled with 'Pareja'. The 'Precio (\$)' field is filled with '3000'. The 'Cantidad (Kg)' field is filled with '1'. At the bottom, there are two buttons: 'Cancelar' and 'Guardar Cambios'.

EDITAR PRODUCTO

Nombre del Producto: Papa Capiro

Fecha de Registro: 22/07/2026

Estado: Activo (Disponible para venta)

Descripción: Pareja

Precio (\$): 3000

Cantidad (Kg): 1

Cancelar Guardar Cambios

Figura 17.

13.4 Historial del Modelo de Predicción (Dashboard)

Permite consultar las predicciones que el sistema calcula automáticamente cada noche, sin necesidad de simular nada.

Pasos:

1. Entra al menú "**Historial**".
2. En el panel de filtros, elige:
 - **Filtrar por Producto:** Todos / Papa Negra / Papa Criolla Bogotá / Papa Criolla Tunja.
 - **Horizonte:** Todos / 1 día / 7 días / 30 días.
 - **Fecha de Predicción:** escribe o selecciona una fecha puntual si buscas un día exacto.
 - **Cantidad:** cuántos registros traer (10, 25, 50 o 100 últimos).
3. Pulsa "**Consultar**" para aplicar los filtros.

4. Usa "**Refrescar**" si quieres traer los datos más recientes sin cambiar los filtros, o "**Exportar CSV**" para descargar la tabla completa.
5. Revisa el gráfico de líneas para ver la tendencia de cada producto en el tiempo, y la tabla inferior para el detalle exacto (precio predicho y rango de confianza al 95 % por fila).

Submódulo — Comparar contra Proyección Diaria Simulada (debajo de la tabla):

1. Elige el **Producto a Simular** y los **Días a Simular** (7, 15 o 30).
2. Ajusta los sliders de Precio, Temperatura, Precipitación, Abastecimiento y Costos si quieres un escenario distinto al que trae por defecto.
3. Pulsa "**Generar y Comparar**".
4. El gráfico "Comparación Individual" te muestra la línea sólida (lo que realmente se guardó en la base de datos) contra la línea punteada (tu simulación) para el mismo producto.

Tip: si un producto no aparece en el gráfico principal más allá de cierta fecha (por ejemplo Papa Criolla Tunja), es porque ese horizonte todavía no tiene modelo entrenado no es un error de la vista.

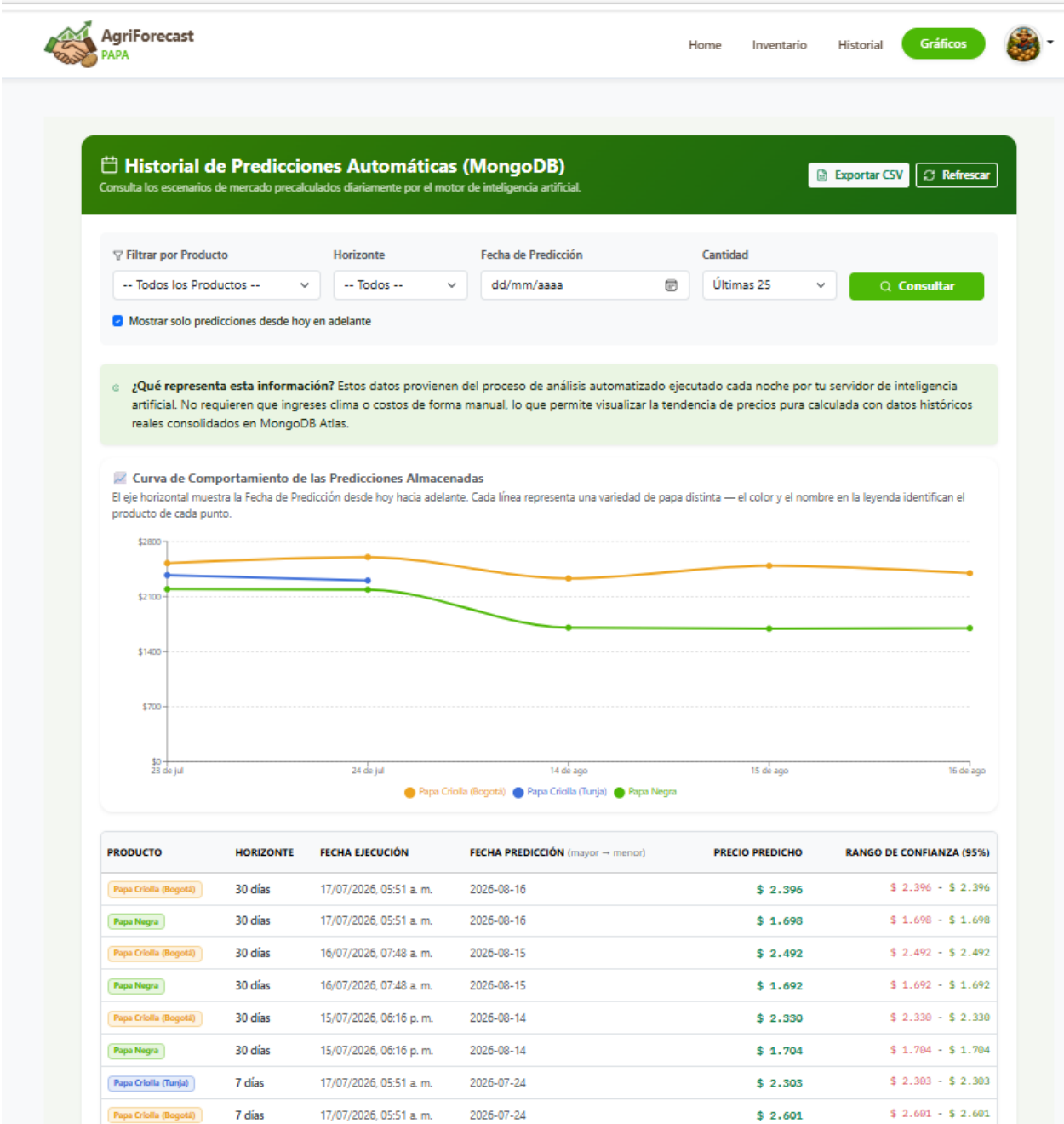


Figura 18.

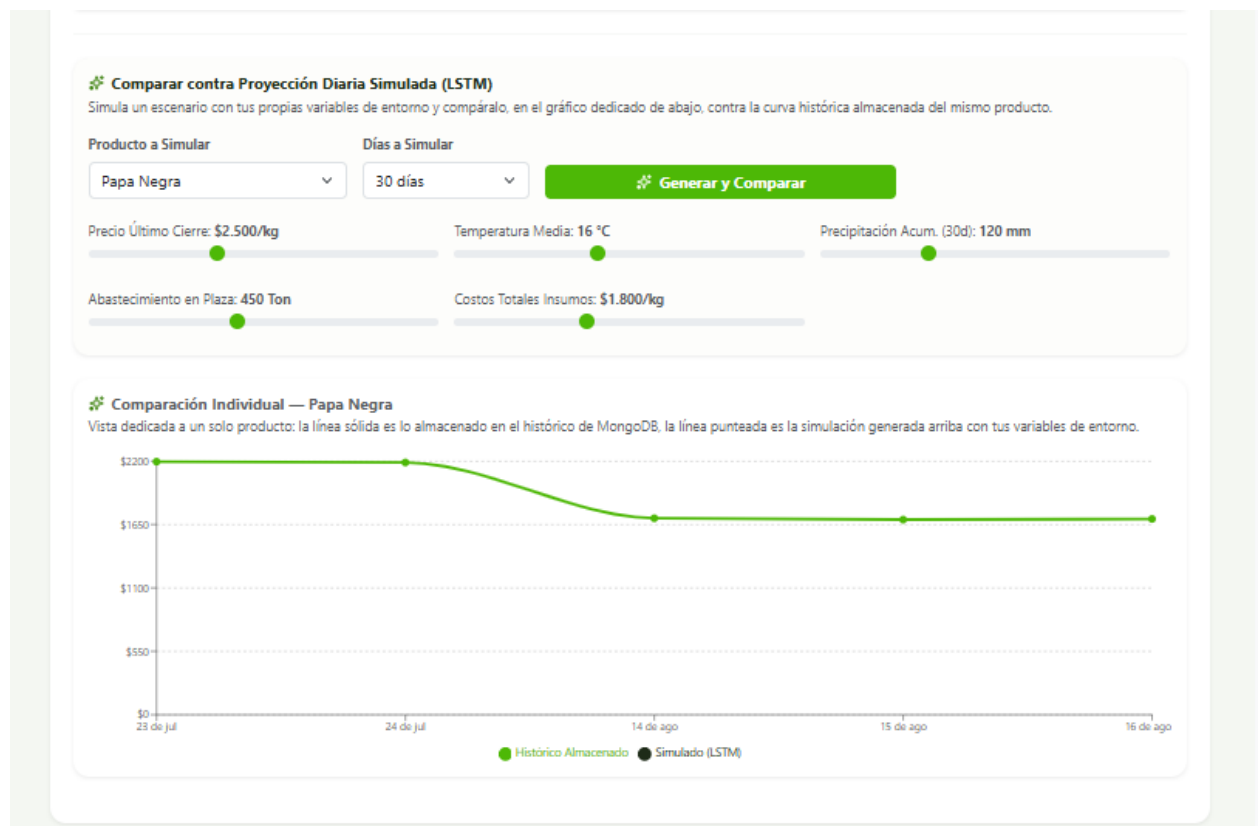


Figura 19.

13.5 Simulador Predictivo Avanzado (LSTM)

Simular el precio con tus propias condiciones (clima, abastecimiento, costos) en vez de solo ver el historial automático.

Paso 1 — Simulación puntual:

1. Entra a **"Gráficos"**.
2. Elige la **Variedad de Papa** (Negra, Criolla Bogotá o Criolla Tunja) y el **Horizonte de Proyección**.
3. Pulsa **"Simular Precio"**.
4. Abajo se cargan los resultados: precio predicho, límites de confianza (95 %), el gráfico "Canal Analytics" y la ficha técnica del modelo.

Paso 2 — Ajustar variables de entorno (opcional):

1. Abre el acordeón **"Configurar Variables del Entorno / Simular Escenario exógeno"**.

2. Verás solo los controles que ese producto y horizonte realmente usan (no todos los productos usan las mismas variables):
 - Precio Último Cierre
 - Temperatura Media (a veces con rezago de 20 días)
 - Precipitación Acumulada (30 días)
 - Abastecimiento en Plaza
 - Costos Totales de Insumos
3. Mueve los sliders y vuelve a pulsar "**Simular Precio**" para ver el efecto de tu cambio.

Paso 3 — Proyección diaria (curva completa):

1. Elige cuántos **Días a Simular** (7, 15 o 30).
2. Pulsa "**Ver Proyección Diaria**".
3. En la curva resultante, los **puntos grandes** son días con modelo propio (1, 7 y 30); los **puntos pequeños** son días interpolados entre esos anclajes — tenlos en cuenta como menos precisos que los puntos grandes.

Cómo leer los paneles de resultado:

Panel	Qué te dice
Canal Analytics	Trayectoria del precio con la banda de incertidumbre sombreada
Ficha Técnica del Modelo	Arquitectura, ventana temporal y pesos de cada variable en la predicción
Brújula del Escenario	Compara tu simulación (verde) contra la condición típica (gris)
Velocímetro de Precisión	Qué tan angosto es el intervalo de confianza — más angosto, más "preciso"
Margen de Ganancia	Compara tu costo de producción contra el precio predicho
Resumen Técnico	Explica en texto simple el resultado de tu simulación

Alerta a tener en cuenta: si Precio Predicho, IC Inferior e IC Superior salen exactamente iguales y el velocímetro marca 100 %, no significa certeza absoluta es una señal de que el modelo no generó suficiente variación en el cálculo interno de esa consulta puntual

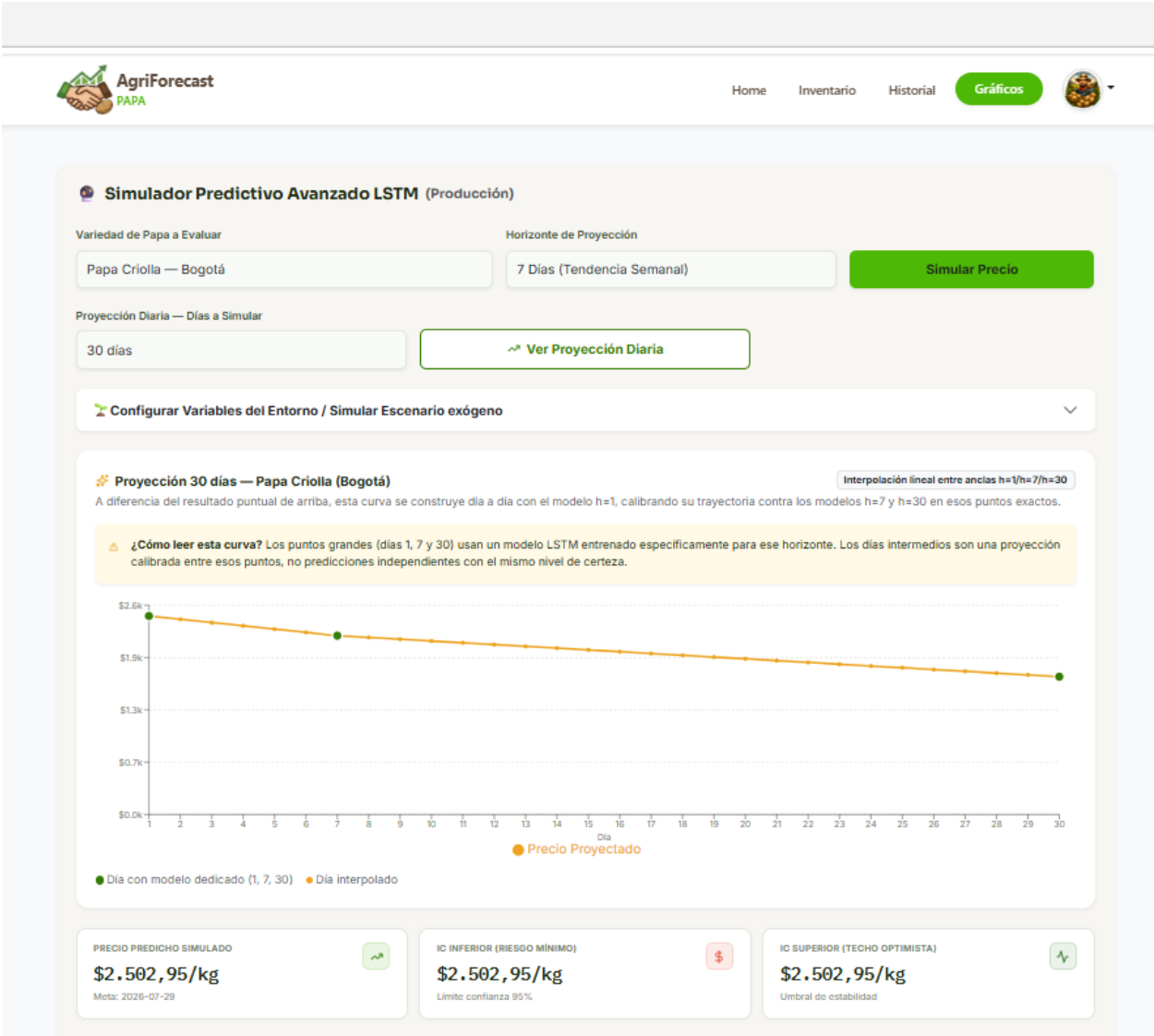


Figura 20.

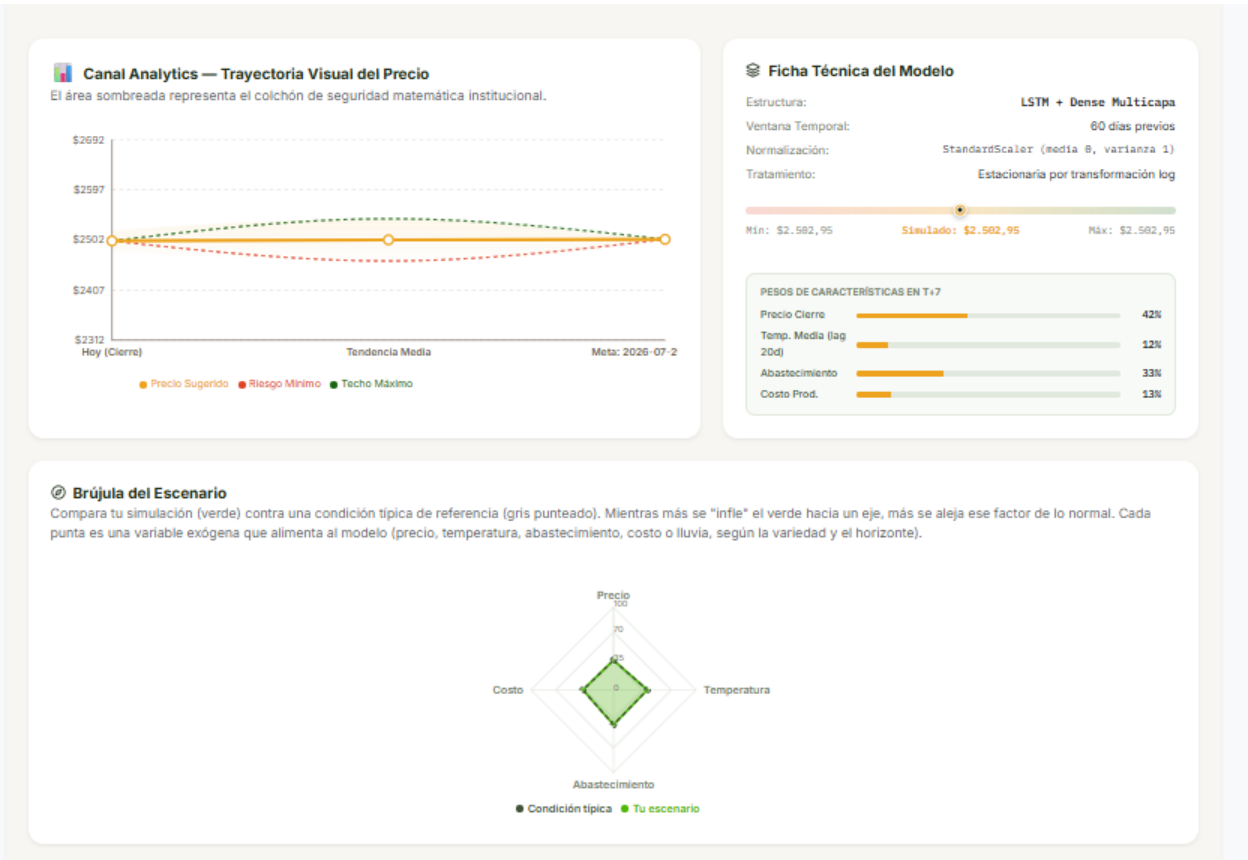


Figura 21.



Figura 22.

13.6 Datos (MongoDB)

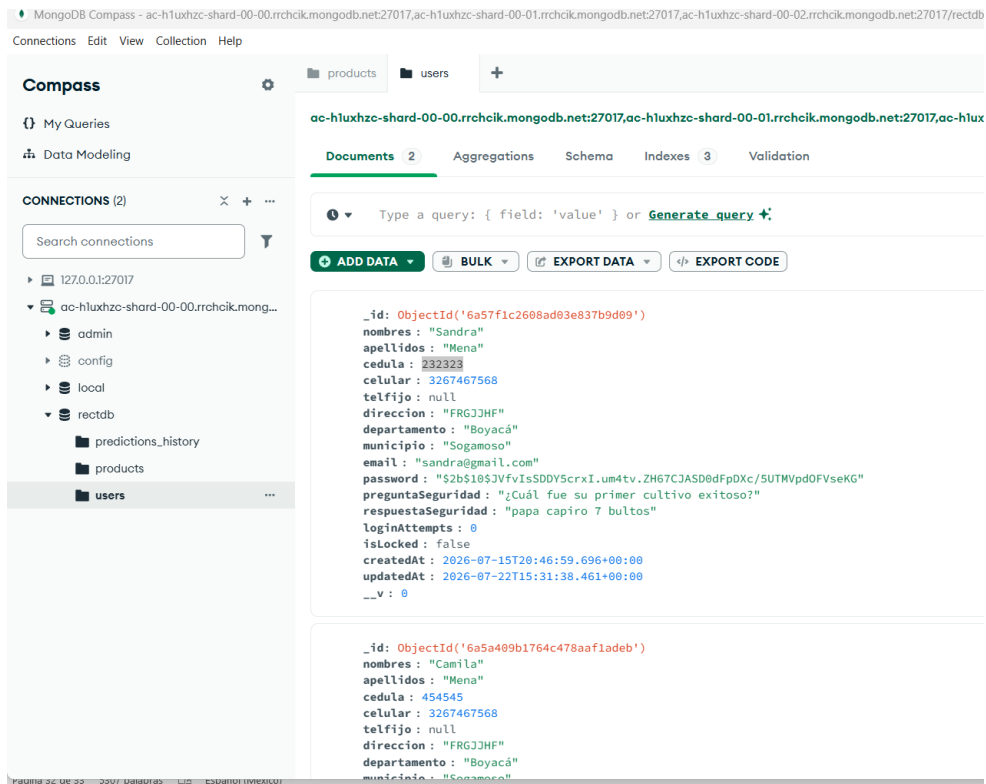


Figura 23.

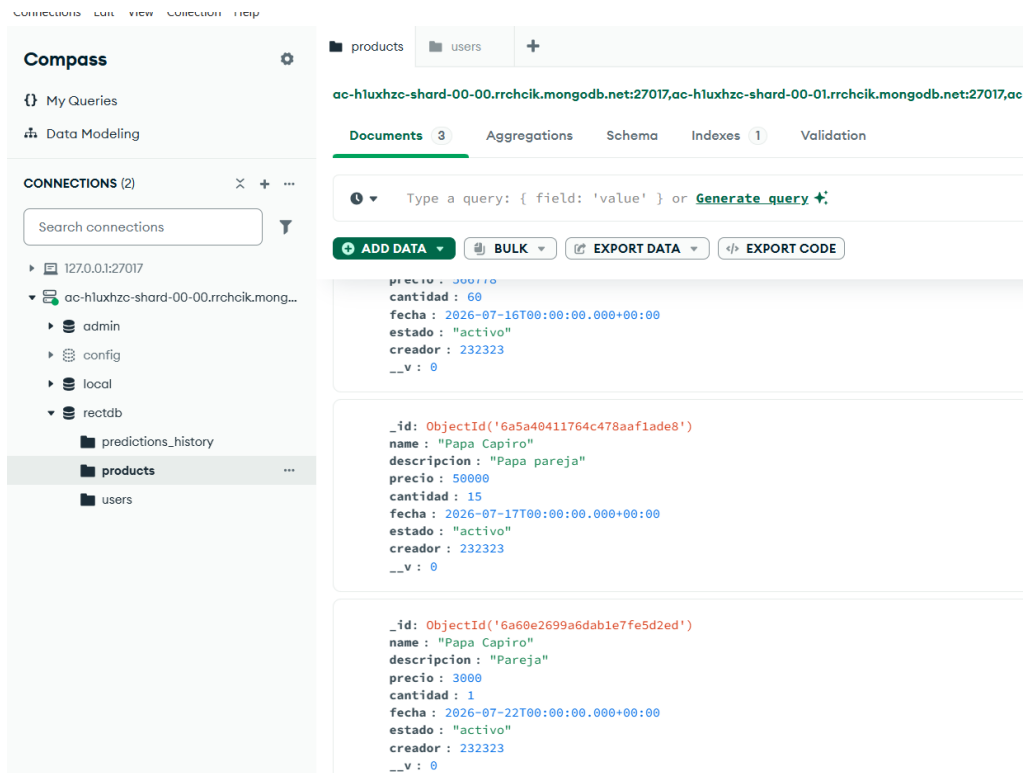


Figura 24.

Apéndice

Código fuente y enlaces de acceso al sistema

Este apéndice reúne la evidencia documental complementaria del proyecto *Modelo Híbrido LSTM con Variables Exógenas para la Predicción del Precio de la Papa en el Altiplano Cundiboyacense*. Se incluyen los notebooks de Google Colab utilizados para el análisis estadístico, el entrenamiento y la validación de los modelos, así como los enlaces al prototipo de software desplegado y al repositorio de control de versiones.

Cada notebook contiene el código fuente completo y comentado, permitiendo reproducir las tablas, figuras y resultados presentados en el documento, en cumplimiento del principio de **reproducibilidad científica**.

- **Notebook 1 — Análisis estadístico univariado:** procesamiento ETL de las fuentes oficiales (SIPSA/DANE, IDEAM, FEDEPAPA) y análisis descriptivo.
- **Notebook 2 — Modelo ARIMA/SARIMA:** línea base estadística con pruebas de estacionariedad, diagnóstico de residuales y pronóstico a 30 días.
- **Notebook 3 — Modelo LSTM-Exógeno (LSTM-E):** arquitectura propuesta, entrenamiento y métricas de error para los ocho pares segmento-horizonte.
- **Prototipo de software desplegado:** acceso al sistema en producción y repositorio en GitHub.

Enlaces de acceso:

- **Notebook ETL y análisis:**
https://colab.research.google.com/drive/1tYJyh7la_wX3lQDRfSmeMEYUtyQdB_gr?usp=sharing
- **Notebook ARIMA/SARIMA:**
<https://colab.research.google.com/drive/1PAWd0LdiJBq6VuX8JnOJp4ujFglex8-T?usp=sharing>
- **Notebook LSTM-E:**
https://colab.research.google.com/drive/1PpErgwY2P0zdWGsFeZ5DQoilwIHTzE_?usp=sharing
- **Prototipo desplegado:**
<https://colab.research.google.com/drive/1PAWd0LdiJBq6VuX8JnOJp4ujFglex8-T?usp=sharing>
- **Repositorio:** <https://pdpreciopapa.onrender.com/>
- **Documento Documentación Técnica de Arquitectura de Software:**
<https://github.com/PAOLAPACHECO2022/PDpreciopapa.git>

Nota: Los notebooks se comparten en modo lectura/comentario; se recomienda solicitar acceso de edición al correo institucional para verificación académica. El prototipo opera en un plan gratuito de Render, por lo que la primera solicitud tras inactividad puede tardar entre 30 y 60 segundos en responder (*cold start*).

Referencias

Efron, B., & Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman & Hall/CRC.

FastAPI. (s.f.). Documentación oficial del framework. Recuperado de <https://fastapi.tiangolo.com/>

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

Internet Engineering Task Force. (2015). RFC 7519: JSON Web Token (JWT). Recuperado de <https://www.rfc-editor.org/rfc/rfc7519>

Meta Platforms, Inc. (s.f.). Documentación oficial de React. Recuperado de <https://react.dev/>

MongoDB, Inc. (s.f.). Documentación oficial de MongoDB Atlas. Recuperado de <https://www.mongodb.com/docs/atlas/>

OpenJS Foundation. (s.f.). Documentación oficial de Node.js y Express.js. Recuperado de <https://expressjs.com/>

Render, Inc. (s.f.). Documentación oficial de despliegue de servicios web, sitios estáticos y variables de entorno. Recuperado de <https://render.com/docs>